# HabitQuest System Design

Complete design reference | README and chapters 01-14

Use the linked chapter references in this opening summary or your PDF reader's bookmarks. Chapter cross-references stay inside this PDF; supporting source links open their original files or websites.

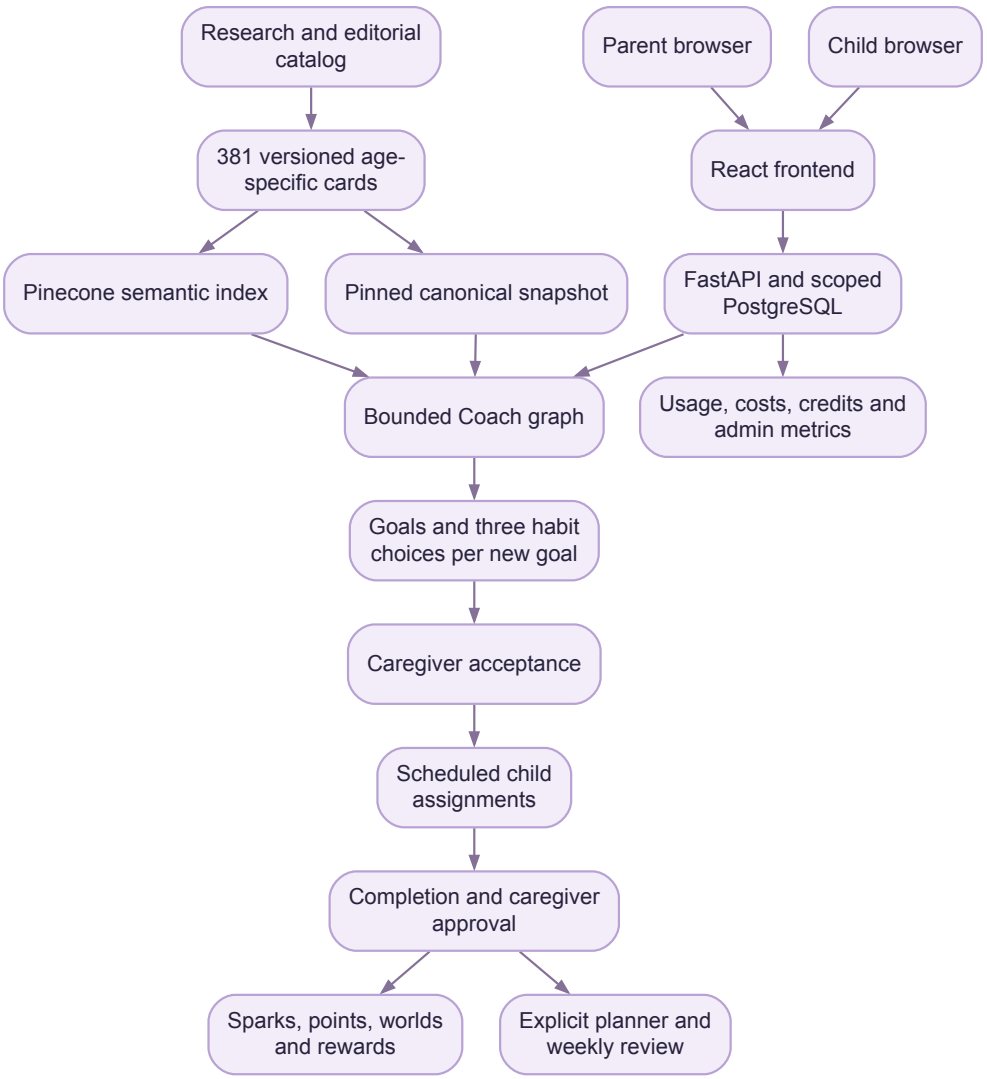
## HabitQuest: current system design

HabitQuest turns a parent's concern into a researched goal, a small set of actionable habit choices, and a child-owned daily quest plan. AI proposes; caregivers choose and approve. The app keeps planning, daily completion, rewards, companions and AI accounting connected without giving a model authority to change a child's plan on its own.

This is the consolidated **November 6, 2026 implemented design** of the main `habbit-quest` repository and its recorded AWS release. It summarizes the final state of the work reviewed here, rather than promising that the design will never change. Current source and completed release evidence take precedence over older proposals. The detailed documentation plan records scope, review responsibilities, document coverage and validation.

The former top-level `docs` is now docs-detailed, preserving all 209 files of investigations, benchmarks, decisions and visual evidence. `backend/docs` remains unchanged. Links to research and deployment sources expect the surrounding HQ workspace; the substantive design is explained here so readers do not need to reconstruct it from that history.

## The application in one picture



The public app is `https://habitquest.quest`; browsers call `https://api.habitquest.quest` directly. Two public ALBs front separate Fargate services. The database is private, always-on RDS PostgreSQL. Research and index publication happen outside user requests.

## 1. System overview and capability boundaries

Read the system overview.

The frontend handles interaction; the backend owns domain rules and approved state. PostgreSQL stores family data. Pinecone finds canonical source cards. Versioned catalog snapshots preserve the evidence and identity behind a goal.

- A catalog card maps to a **goal**; a generated action maps to a **habit**.
- An **assignment** schedules a habit for one child; generating a card alone does not assign it.
- The legacy seeded catalog, imported researched goals and generated habits coexist.
- Browser state is a projection of backend state, not a separate authority for points or plans.
- Research, operational generation and infrastructure deployment are separate processes.

## 2. Frontend and family workflows

Read the frontend experience.

React/Vite provides separate parent, kid and operator experiences. Parent flows cover family setup, child details, Coach, plan management and approvals. Kid flows show actionable instructions, today's quests and progress without exposing adult guidance or sibling data.

- Coach combines child selection, compact budget meter, chat, stages and location icons.
- New goals appear before older recommendations and the saved-plan section.
- Three habit choices start automatically for eligible new goals, with at most two batches in flight.
- Quest Library keeps **Quests** as its default and offers **By goal** for the exact saved relationships.
- Dashboard combines Plan and Today, so unscheduled-for-today assignments still appear in the child's plan.
- Small cards open richer details; goal research, age/context and original artwork remain available.

## 3. Backend, identity and security

Read the backend/API design.

FastAPI validates requests, verifies identity, performs business operations and enters scoped PostgreSQL transactions. Authentication and database isolation back up frontend route guards. Caregiver, kid and operator identities are distinct.

- Google ID tokens are verified before issuing HabitQuest caregiver tokens.
- The current Google flow uses a public client ID, not a runtime client-secret exchange.
- Kid credentials scope actions to one child; caregiver membership scopes family access.
- Optional parent PIN confirmation is distinct from a child's login PIN and admin credentials.
- `hq_api`, `hq_kid`, `hq_meter` and `hq_admin` have different database responsibilities.
- AI request IDs, reset fences, row locks and revision checks protect retries and concurrent actions.

## 4. Database and daily quest model

Read the database and domain model.

PostgreSQL separates source goals, child goals, generated draft batches, approved habit instructions and scheduled assignments. That separation supports precise goal links, parent review, safe removal and retained completion history.

- The migration set defines 36 application tables plus the migration ledger across 22 files.
- Six non-removed assignments fill a child's current capacity; paused habits still occupy a slot.
- Accepted generated instructions retain ownership and cannot be silently regenerated in place.

- Child submissions and caregiver approval are separate states.
- Sparks track progress; points support rewards. Coaching credits are a separate AI allowance.
- Row-level policies, constraints and narrow database functions protect cross-family and cross-child boundaries.

## 5. Agentic AI architecture

Read AI orchestration.

Agentic design here means an explicit, bounded workflow with structured outputs, local tools/checks and controlled model invocations. LangGraph orchestrates the stages; LangChain and the imported planner adapters handle provider calls. A graph node does not automatically mean an independent LLM agent.

- The current refined Coach replaces the old multi-role crew as the configured path.
- Normal Coach uses one central generation plus one review of newly generated prose.
- Name masking, eligibility, canonical checks, context assembly and persistence run in code.
- Retrieval uses one embedding plus Pinecone search, without a research-generation or catalog Safety call.
- One bounded prose revision is permitted; SDK retries are counted separately from logical calls.
- Habit writing and explicit planner/review operations are separate from the Coach graph.

## 6. Habit Coach and generated habits

Read the complete Coach/habit workflow.

Coach retrieves suitable source cards, creates reviewed guidance and returns ordered goals. Only selected reviewed goals are materialized locally. The UI then prepares small habit batches for the selected child and waits for caregiver acceptance.

- Up to four returned goals can receive three choices each; usable saved choices are reused.
- The writer normally makes one call for a batch, followed by local source/schema/age/action checks.
- One correction may repair a validation failure; there is no unbounded retry or whole-habit Safety loop.
- “Get more goals” and “More habit choices” preserve earlier choices and require explicit action.
- Research supports/limitations belong to the source goal; exact generated habits do not inherit an efficacy claim.
- The November 6 settlement fix keeps successful recommendations chargeable and their automatic slots intact.

## 7. Other AI capabilities

Read planner, review and other AI.

Quest Planner and Weekly Review run when requested, using the family's existing history and returning proposals. They are not hidden background stages on every Coach turn. Other entry points have their own data and safety contracts.

- Planner proposes plan changes; Weekly Review summarizes patterns and suggests next steps.
- Schedule edits and proposal acceptance are database operations, not model calls.
- Legacy onboarding mapping has its own structured generation/fallback path.
- Optional legacy buddy prose and child-note screening use their established screening paths.
- Catalog buddy selection and catalog event lines are deterministic and local.
- Review behavior is capability-specific: do not assume every helper has the same extra Safety call as Coach.

## 8. Model decisions and runtime routing

Read model selection and routing.

The implementation assigns models by role and keeps public routing configuration separate from provider secrets. Selection considered structured-output behavior, integration compatibility, measured useful results and cost-not only published model throughput.

- OpenAI `gpt-6-luna` is configured for Coach writing and generated habit batches.
- Nebius `deepseek-ai/DeepSeek-V4.1-Flash-1017` serves generated-text Safety and the other configured helpers.
- OpenAI `text-embedding-3-small` produces 512-dimensional retrieval vectors.
- Report-helper reasoning settings address the observed DeepSeek response-budget failure.
- The initial Nebius shortlist and one-total-call proposal remain historical decision inputs.
- Model/provider/role/API-mode cohorts matter; the five-second objective is not a production latency guarantee.

## 9. Embedding research and selection

Read the embedding selection study.

The study compared lexical retrieval and eight embedding/dimension configurations using frozen synthetic queries, eligibility rules and editorial relevance labels. The executable results live under HQ infrastructure's embedding evaluation; the broader behavioral research lives under HQ research.

- The study used 405 documents:381 habit cards and 24 buddies, with separate candidate sets.
- Large/512 was the original quality-first selection; small/512 was the owner's cost/quality decision.
- Small/512 and large/512 had the same first-useful-result MRR, while large/512 had stronger top-five coverage.
- The study did not establish matched API latency superiority for small/512.
- The live index contract fixes small/512/cosine; changing embedding geometry requires a coordinated rebuild.
- Buddies remained local because 24 choices do not justify an embedding dependency for ordinary selection.

## 10. Habit research and Pinecone publication

Read research, catalog authoring and Pinecone.

Behavioral evidence and books informed editorial habit concepts, adaptations and age-specific cards. Cross-review and deterministic validation prepared those cards for public catalog browsing and later semantic search.

- The catalog contains 96 concepts across 24 categories, expanded to 381 age-specific cards.
- The evidence set contains 89 research records and 6 books, with population and applicability limits.
- Cards include observable actions, parent support, categories, age/context and source references.
- Offline ingestion validates canonical hashes and actual small/512 vectors before publishing to Pinecone.
- Runtime retrieval validates returned IDs/metadata against the packaged canonical snapshot.
- Pinecone stores public source records, while PostgreSQL owns family plans and generated actions.

## 11. Tokenomics and generation observability

Read economics, credits and timing.

Durable operation/attempt records separate useful family activity from provider spend. Costs include chat tokens, embeddings and Pinecone usage. Timers distinguish server stages, provider attempts, browser response time and all-choices readiness.

- Seeded policy: 100 daily credits,2 per useful Coach/planner/review/mapping,1 per useful habit batch.
- Demo mode tracks usage beyond allowance without enforcing it; policy edits are audited and revision-checked.
- Retries/failures retain operator exposure even when they do not create a useful-result credit debit.
- Admin history shows input/output tokens with measured/partial/unknown distinctions.
- Coach keeps the compact allowance chip and omits inline credit/preparation copy.
- Historical benchmarks are dated samples; no invoice reconciliation or representative cloud p95 is claimed.

## 12. Buddies, graphics and presentation

Read buddies and artwork.

Companion selection uses local metadata and explicit choices. Catalog art improves recognition and appeal without changing the underlying wardrobe or reward rules.

- Every child has a default or saved buddy; there is no empty buddy option.
- Twenty-four catalog characters supplement four original SVG buddies.
- Suggested age bands are soft; all current catalog buddies support ages 3-13.
- Name, characteristics and suggested ages appear during selection, with compact shortlists and a full chooser.
- Existing accessories use per-character overlay anchors on the generated artwork.
- Goal/Fluent assets ship with the frontend; versioned buddy PNGs load from public S3 URLs.

## 13. AWS deployment and operations

Read the final AWS design.

The demo runs continuously in `us-east-1`: two public ALBs, separate frontend/API Fargate services and private RDS PostgreSQL. Source-hashed images and checked CloudFormation change sets make releases reviewable and repeatable.

- GoDaddy registration delegates to Route 53; shared ACM HTTPS serves apex, API and redirect hosts.
- Public-IP tasks use the internet gateway for egress; inbound task ports accept only their own ALB group.
- RDS is always-on micro, Single-AZ, with scoped SQL access and verified TLS.
- ECS injects backend secrets from Secrets Manager; the browser receives public build configuration only.
- Builds, audits, synthetic container/browser tests, immutable ECR publication and rollout receipts are documented.
- The recorded current release passed public health/auth-boundary checks; provider generation and HA limits are stated separately.

## 14. Design process, validation and remaining limits

Read the decision and verification record.

The design evolved through research, retrieval experiments, catalog/prototype work, main-app integration, call-stack simplification, plan/visual fixes, durable accounting and staged deployment. The documentation preserves why choices were made and where evidence is incomplete.

- Earlier alternatives remain in `docs-detailed`, clearly separated from the implemented path.
- The latest application baseline passed 1,073 backend tests at 100% statement/branch coverage and 206 frontend tests.
- Fresh disposable-database/container checks covered identity, child isolation, plans, assets and fake-provider flows.
- Documentation validation checks links, diagrams, retained files and consistency without making new provider calls.
- Outstanding limits include production latency sampling, exact Pinecone billing assumptions and retained base-image findings.
- Future work should update this index and its capability documents while retaining dated evidence.

## Suggested reading paths

- **Product/design:** overview -> frontend -> Coach -> buddies -> research.
- **Backend/data:** API/security -> database -> AI orchestration -> Tokenomics.
- **AI/research:** orchestration -> Coach -> other AI -> model routing -> embeddings -> catalog pipeline.
- **Deployment/demo:** AWS -> validation/decisions -> linked deployment runbooks.

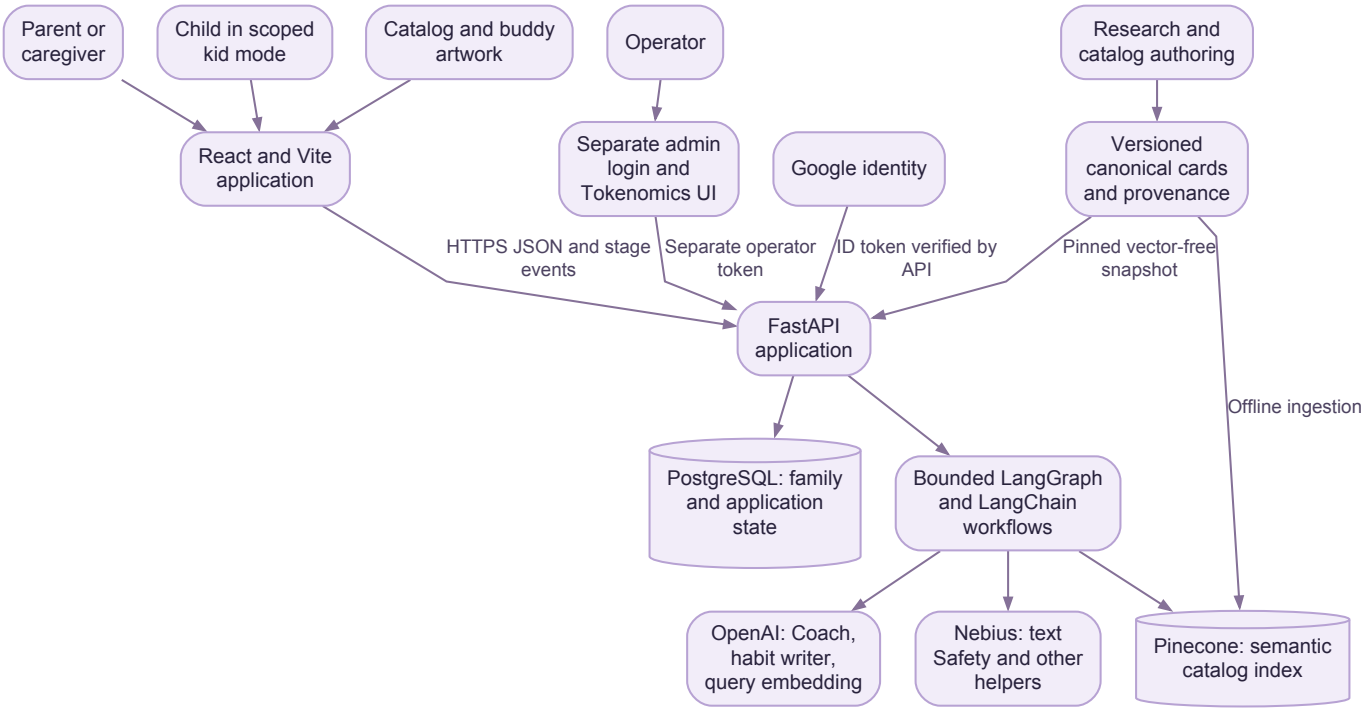
The complete work plan and maintenance contract are in 00-documentation-plan.md.

# 01 System overview

HabitQuest helps parents turn a concern into a researched goal, choose small habits for a child, and reinforce completion through a daily quest experience. AI proposes guidance and choices. Parents decide what enters the plan; the database controls ownership, schedules, approvals and rewards.

This is the implemented November 6, 2026 design. It combines the current `habbit-quest` application with research/catalog assets and the recorded AWS release. The original standalone backend and multi-agent crew documents describe earlier stages, not the default deployed request path.

## System context




The browser communicates directly with the public API origin. Serving the SPA from a frontend container does not make that container an API proxy. CORS, API tokens and database authorization remain necessary. The deployed network is described in AWS deployment.

## Components and ownership

| Component | Responsibility | Does not own |
|---|---|---|
| React frontend | Parent/kid/admin navigation, local display state, progress, choice/approval controls | Provider credentials, authoritative rewards or credit settlement |
| FastAPI backend | Identity, family/child authorization, business rules, AI invocation and result persistence | A continuously running autonomous planner |
| PostgreSQL | Accounts, plans, immutable approved instructions, completion, reward ledgers, AI operations and policy | Vector similarity ranking |
| Canonical catalog snapshot | Stable source IDs, age/context metadata, research provenance and integrity | A family's current plan |
| Pinecone | Find relevant canonical source cards from a masked query embedding | Family conversations, assignments or generated habit ownership |

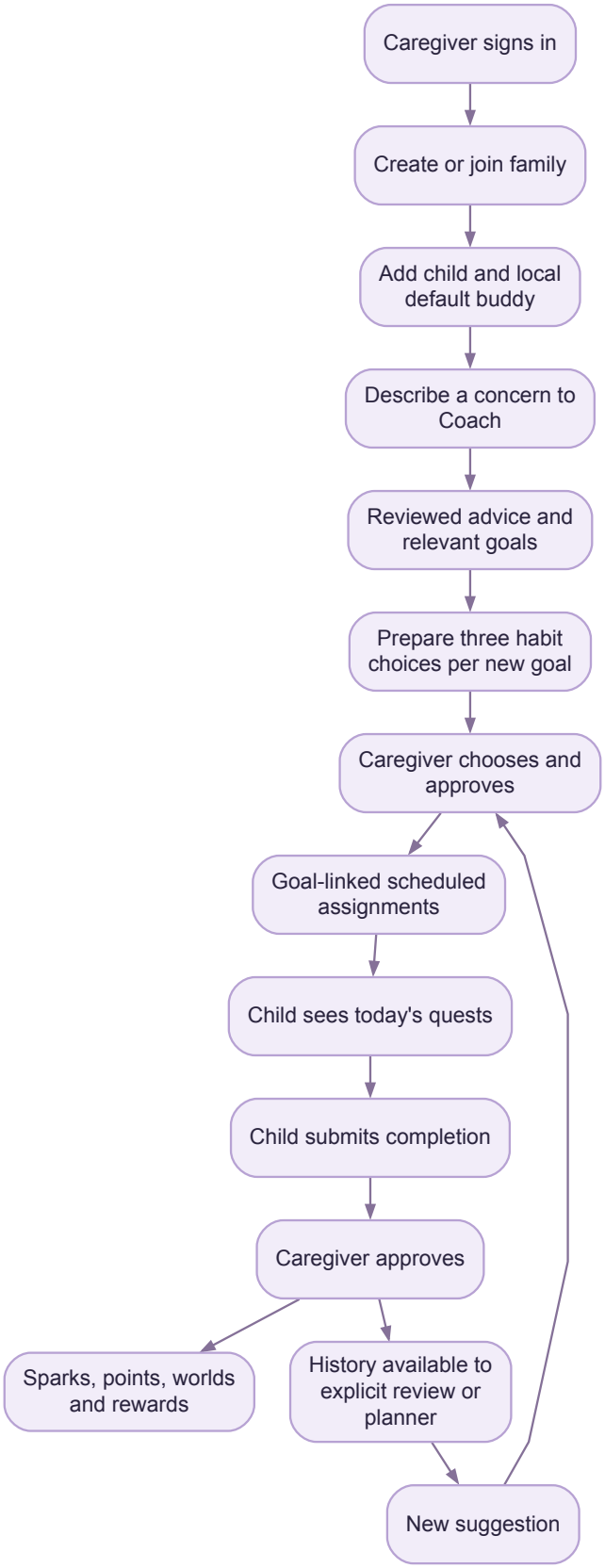
| Component | Responsibility | Does not own |
|---|---|---|
| Chat providers | Produce bounded structured guidance/actions/reports and review eligible new text | Direct database writes or automatic assignment to children |
| Artwork catalog | Goal reference images, fluent icons, buddy illustrations and attachment anchors | Model-based personality inference |
| AWS deployment workspace | Infrastructure templates, builds, public configuration, secret references, execution evidence | Application secrets embedded in Markdown or browser bundles |

## The domain vocabulary

| Term | Meaning |
|---|---|
| Research source | Evidence or reference with applicability and limitations |
| Canonical card | Versioned age/context source record; Pinecone returns its search projection and the full record is hydrated from packaged canonical JSON |
| Goal | Main-app representation of that canonical card; imported when selected by reviewed generation |
| Draft habit | Generated action beneath an exact goal, scoped to a child and awaiting acceptance |
| Approved habit | Persisted instructions accepted by a caregiver; later generation cannot silently rewrite them |
| Assignment / quest | That habit scheduled in the child's plan, with days, cue time and status; setting belongs to the source/draft generation context |
| Suggestion | A proposed addition or plan change, still waiting for caregiver action |
| Sparks / points | Separate progress and reward systems derived from approved completions |
| Coaching credits | Family AI allowance units; distinct from reward points and actual provider dollars |

The legacy seeded catalog coexists with imported researched goals and generated habits. The system does not bulk-import every Pinecone card into family plans. See the database design.

## End-to-end family workflow




The current plan allows six non-removed assignments per child; paused entries still occupy a slot. Removing a habit releases capacity while preserving history. Goals may exist without an assigned habit. Displaying a generated card does not mean it was accepted. Dashboard, quest library and goal detail views resolve the same saved assignment and exact goal relation.

## Frontend and backend workflow boundaries

| Activity | Frontend responsibility | Backend responsibility |
|---|---|---|
| Sign-in | Google identity UI or development-only login; hold the appropriate session | Verify identity and issue scoped application token |
| Child setup | Capture age/routine and present local buddy recommendations | Validate family ownership and persist child/buddy revisions |
| Coach | Send explicit request, show stages and recommended cards, queue at most two habit batches | Mask names, retrieve, generate/review prose, validate source IDs and persist result |
| Habit preparation | Request three choices, retain earlier choices, show small cards and detail modal | Produce a bounded batch, validate it locally, save child-bound drafts |
| Plan changes | Ask for caregiver confirmation/PIN where configured; refresh affected views | Enforce capacity, source/child ownership, status and transactional linking |
| Kid completion | Show child-facing instructions and submit completion | Enforce child/day/state scope; caregiver approval applies rewards |
| Review/planner | Explicit dashboard action; show proposed changes | Summarize history or produce plan proposals without applying them automatically |
| Administration | Separate login, costs/usage and policy editing | Separate operator identity, restricted grants, audited revision-checked configuration |

## What agentic design means here

The main app uses a finite LangGraph to coordinate named stages and LangChain adapters for structured model calls. Most work is deterministic code: ownership checks, context collection, catalog integrity, plan eligibility, serialization, validation, accounting and persistence.

Normal refined Coach uses one central generation and one review of new prose. It does not run the old triage/research/progress/planner/fit crew on each turn. Retrieval itself uses an embedding and Pinecone search, without a chat-model research or Safety call. Habit writing, Quest Planner and Weekly Review are separate operations. See AI orchestration and the capability inventory.

## Design process

1. Research established practical habit concepts, age-specific cards and evidence limits.
2. Embedding experiments measured retrieval quality/cost tradeoffs before fixing the index contract.
3. The catalog/planning prototype demonstrated retrieval plus generated actions and richer visual cards.
4. Main-app integration mapped canonical cards to goals and actions to habits, preserving the daily quest loop.
5. Workflow review reduced serial model work, retained generated-text review and separated explicit planning.
6. UI and database audits corrected plan visibility, source links, capacity handling and child-facing detail.
7. Tokenomics added durable usage/cost accounting, credits, stage timings and operator controls.
8. Container, database and network rehearsals preceded the two-ALB AWS HTTPS deployment.

This process produced tradeoffs, not universal guarantees. The selected embedding model was cheaper, not the winner on every retrieval metric. Five-second responses remain a goal, not a measured production p95. Current validation and remaining limits are in design decisions.

## Sources and next reading

- Frontend entry and routing, state synchronization
- API factory, database transactions

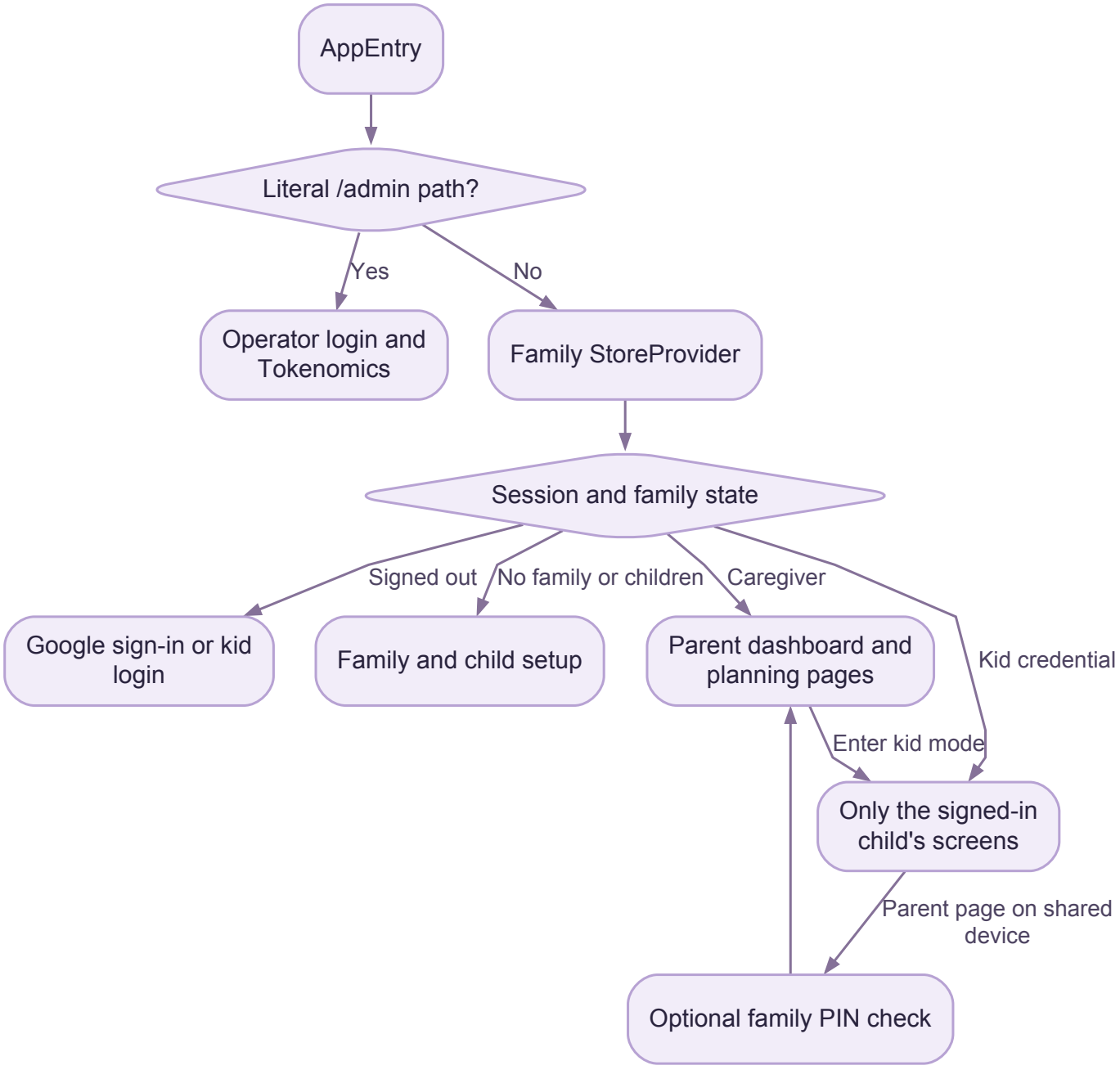
- Coach, refined graph
- Habit generation, daily loop
- Research and Pinecone, deployment record

# 02 Frontend experience and application workflows

This document describes the implemented React/Vite app as of November 6, 2026. The browser is a presentation and interaction layer over the backend. It does not independently award points, decide ownership, or turn an AI suggestion into a child's assignment.

## Application shell and audiences

The family application uses hash routes. The literal `/admin` path mounts a separate operator application before the family store is created. Parent and kid screens share visual primitives, but use different API credentials and different server projections.



| Surface | Route | Main responsibility |
|---|---|---|
| Landing, privacy and login | `#/`, `#/privacy`, `#/login` | Explain the product, authenticate and link to operator login |
| Family joining and onboarding | `#/join/:code`, `#/welcome` | Distinguish grown-up invite access from kid code/PIN access; create a family and its first child |
| Dashboard | `#/parent` | Approvals, each child's complete assigned plan, progress, check-ins, rewards and AI helpers |
| Child details | `#/setup`, `#/setup/:child` | Profile, age, routine times, buddy and relevant child settings |
| Habit Coach | `#/coach`, `#/coach/:child` | Conversational advice, recommended goals, habit choices and explicit acceptance |
| Quest Library | `#/quests/:child` | Manage the child's assignments in the default Quests view or By goal view |
| Exact goal view | `#/quests/:child/goal/:goalId` | Display only the goal and assignments with that exact saved goal ID |
| Family and rewards | `#/family`, `#/rewards` | Caregivers, codes/PINs, data controls, shop items and time limits |
| Child experience | `#/kid/:child`, `#/world/:child/:world`, `#/shop/:child`, `#/journal/:child` | Today's quests, worlds, rewards, wardrobe and field journal |
| Operator console | `/admin` | Separate authentication, Tokenomics policy, price inputs and sanitized generation metrics |

Sources: app routing, operator entry boundary, family state, operator page.

## State ownership and synchronization

The server owns families, children, plans, completions, approvals, points and saved generated habits. `StoreProvider` loads an account/family snapshot and provides the browser view consumed by pages. `sync.ts` applies authoritative Today, progress and extras responses to that view. A cached snapshot helps boot and shared-device continuity; it is not a second writable database.

- Caregiver credentials and account snapshots use browser local storage.
- A caregiver handing over a device uses a per-tab kid session and kid-mode flag.
- A child's own code/PIN login has a separate stored kid identity.
- Optional generation timing preferences/history are browser-local and account scoped; durable generation accounting belongs to the backend.
- Coach history has an in-memory child/account cache. Saved messages and choices are fetched from the API; passive reads do not request new generations.
- Buddy selection revisions prevent a delayed Today/extras response from replacing a newer explicit selection.

The API client uses bearer authentication and separates normal request deadlines from AI request deadlines. It can retry eligible reads while a server wakes, but does not blindly replay writes after gateway/network failures: a write may already have committed. Generation request IDs and saved operation results provide the appropriate replay boundary instead.

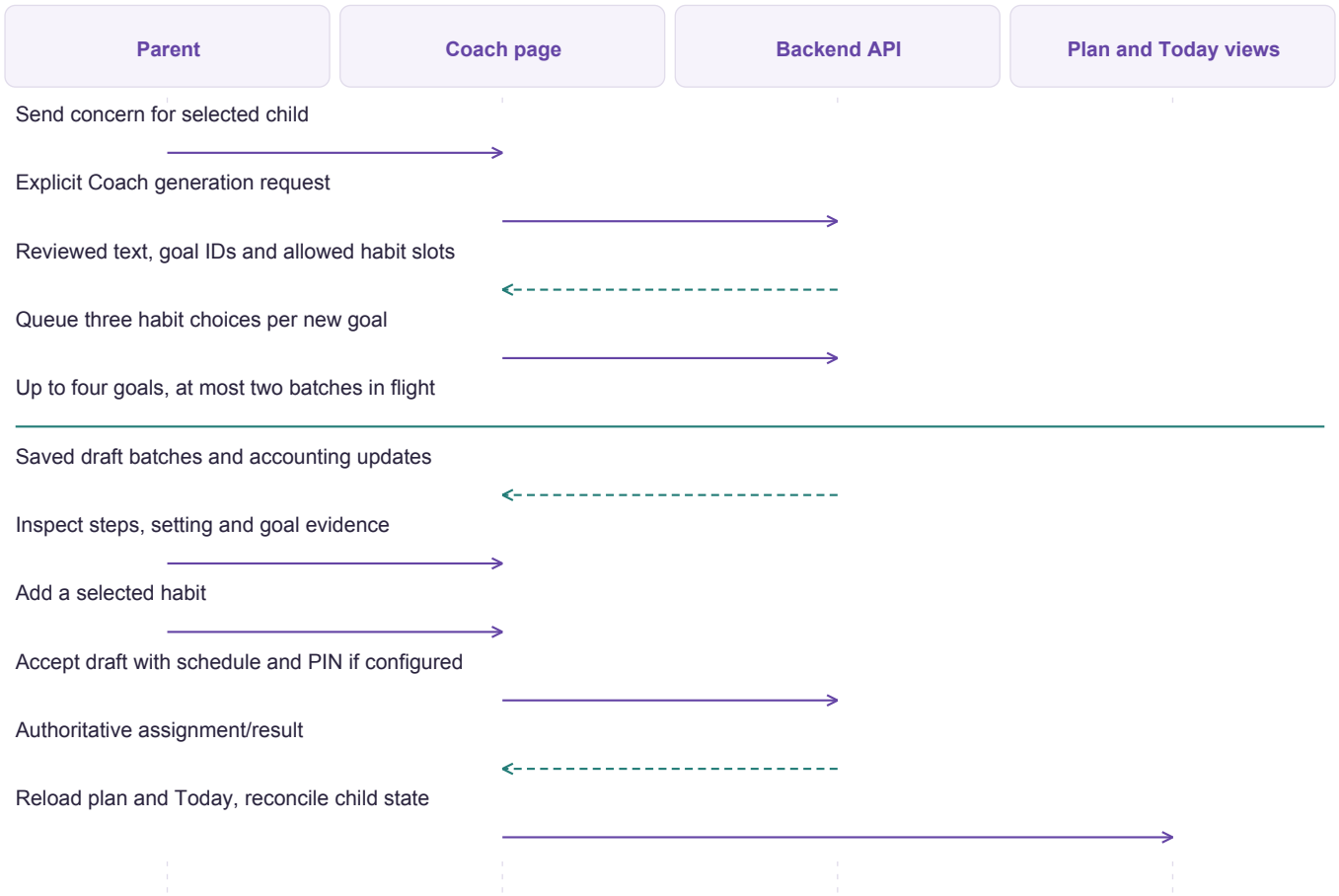
Sources: storage, sync, API client, Coach cache, generation preferences.

## Parent planning journey

The app deliberately separates five things that may look related on screen:

1. **A recommended goal** is a source-backed suggestion from Coach.
2. **A saved goal** is a goal the caregiver has put in this child's plan.
3. **A habit choice** is a generated draft beneath that goal.
4. **An accepted habit** has immutable reviewed content and an owning child.
5. **An assignment** schedules that habit on chosen days and at a cue time.

Generating choices does not assign them. Saving a goal does not silently add every habit under it. Details use the persisted goal ID, rather than guessing the goal from a category or similar title.




Coach keeps the child selector, compact credit chip, chat, stage feedback and small setting icons together. A new response's goals appear before earlier recommendations and the saved-plan section. A saved goal recommended again is shown once with its saved status. Goal cards use compact habit rows, with larger dialogs for steps, grown-up guidance and research. “Get more goals” appends goals; “More habit choices” requests another bounded batch without deleting prior choices.

The current Coach hides inline per-response/prepared-habit credit copy. Its compact daily meter remains. The November 6 credit correction also preserves automatic habit slots for successful refined responses. See Coach and generation design and Tokenomics.

Sources: Coach, goal cards and preparation queue, accepted habit details, setting inference.

## Dashboard and Quest Library

The dashboard fetches **both Plan and Today** for each child. Today is a scheduled daily projection; it cannot stand in for the full plan. This distinction fixes the earlier appearance that newly assigned habits had vanished on non-scheduled days. Habit rows precede the Goals summary. The dashboard refreshes after mutations, on focus/visibility and on a 30-second visible-page polling interval.

Quest Library has two views of the same data:

- **Quests** is the default list with schedule, pause/resume, removal and details.
- **By goal** groups the same assignment rows beneath an illustrated goal summary. It reuses assignment controls; changing the view creates no new plan records.
- Direct goal links filter by exact `goal_id`, and source-goal details validate that the requested goal belongs to the child's accessible plan.

The current limit is six non-removed assignments per child; paused habits still occupy capacity. Draft choices do not. A full plan offers a removal path and keeps generated choices available. Removing a habit retains its history; restoration uses existing approved content instead of spending on another generation.

Sources: dashboard, Quest Library, goal details, plan capacity and restoration.

## Child loop, progress and rewards

The child sees picture quests and simple instructions. Generated-habit details include steps, materials, cue, help and a recognizable goal image. Grown-up preparation, conversation content and parent guidance are excluded from the kid API projection.

1. The child submits “I did it!” for an assigned quest.
2. It becomes pending, not immediately approved or rewarded.
3. A caregiver approves or sends it back. Rejected submissions can be retried.
4. The backend returns the new points and journey progress; the UI celebrates those returned values.
5. Spendable points support the reward shop and requests. Sparks drive worlds and are not spent on shop items.

The journey includes six worlds, landmarks, quest stars, treasure chests and eight wardrobe accessories. Check-ins, caregiver surprises and the daily app-time limit are separate features. Buddies are persistent identities with local artwork and deterministic selection, described in buddies and assets.

Sources: kid screen, quest steps, daily rules, journey, rewards.

## Design process and verification boundaries

The interface evolved from broad text cards into goal-led, illustrated choices with compact previews and focused detail dialogs. The important corrections were data-flow corrections as well as styling: exact goal links, Plan-versus-Today separation, explicit assignment, child-safe detail projections and stale-response guards. Regression journeys cover two children, multiple goals, approval, chat reset and restoration.

Screen tests use a fake API; browser journeys use isolated synthetic databases. Those checks establish UI and persistence behavior, not live-model quality or a latency guarantee. Production uses built static assets and does not hot-reload already-open tabs as local Vite does. A browser refresh loads a new deployed build.

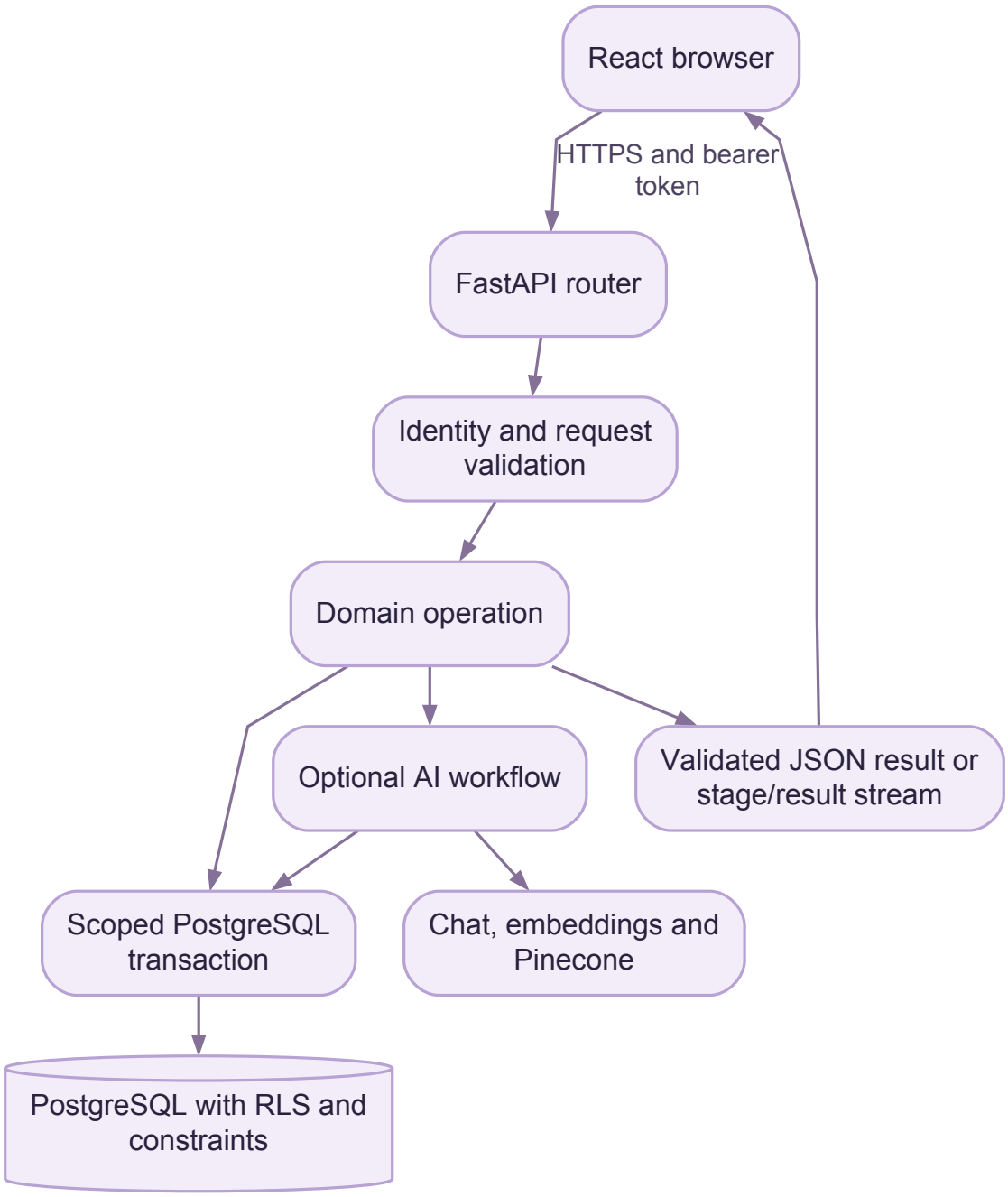
Further evidence: planning screen tests, goal-link tests, library view tests, kid-mode tests, historical UI decisions.

# 03 Backend API, authentication and access boundaries

HabitQuest's implemented backend is one FastAPI application with domain-focused Python modules, PostgreSQL transactions and LangChain/LangGraph AI workflows. It is not a set of independently deployed microservices. This is the November 6, 2026 design; the AWS runtime is described in deployment.

## Request structure

`create_app` loads validated settings, configures allowed browser origins, installs health endpoints and registers feature routers. Pydantic request models validate input shapes; protected endpoints obtain caregiver, kid or operator identity through explicit dependencies.



| Module group | Responsibility |
| --- | --- |
| `accounts`, `auth`, `limits` | Google/token exchange, families, caregivers, codes, PINs, kid identities and attempt limits |
| `onboarding`, `habits` | Children, routine times, source/catalog access, goals, assignments and custom habits |
| `daily`, `journey` | Today, submissions, approvals, points multipliers, Sparks, worlds and quest stars |
| `rewards`, `extras`, `checkins` | Reward shop/requests, wardrobe/chests, family settings, surprises, check-ins and safety alerts |
| `coach`, `refined`, `generated_habits` | Coach conversations, reviewed goal suggestions, drafts and explicit acceptance |
| `planning_catalog`, `retrieval` | Pinned canonical source binding and semantic retrieval |
| `insights`, `ai`, `safety` | Separate planner/review, onboarding mapping, optional buddy lines and child-text screening |
| `generation_progress`, `generation_metering`, `tokenomics*` | Stage transport, durable generation admission, usage/costs and family credit views |
| `admin` | Operator-only configuration and sanitized metrics |

Source: application factory, configuration model.

## Three identities, three application boundaries

### Caregivers

Google Identity Services gives the browser a Google ID token. The browser sends that credential to `POST /v1/auth/google`. The backend verifies the RS256 signature against cached Google public keys, issuer, expiration, configured client-ID audience and verified-email claim. It resolves the local user and returns HabitQuest's own HS256 bearer token.

The real Google sign-in endpoint issues an **eight-hour** session, as do kid session/login endpoints. The generic token helper's default is 15 minutes; it is not the duration used by those endpoints. A user can be a caregiver in more than one family through `caregivers`. Invite codes join grown-ups to a family.

This flow does not use a Google client secret at runtime. The browser receives only the public client ID. Development username/password login is a separately gated feature and is disabled in the deployed public app.

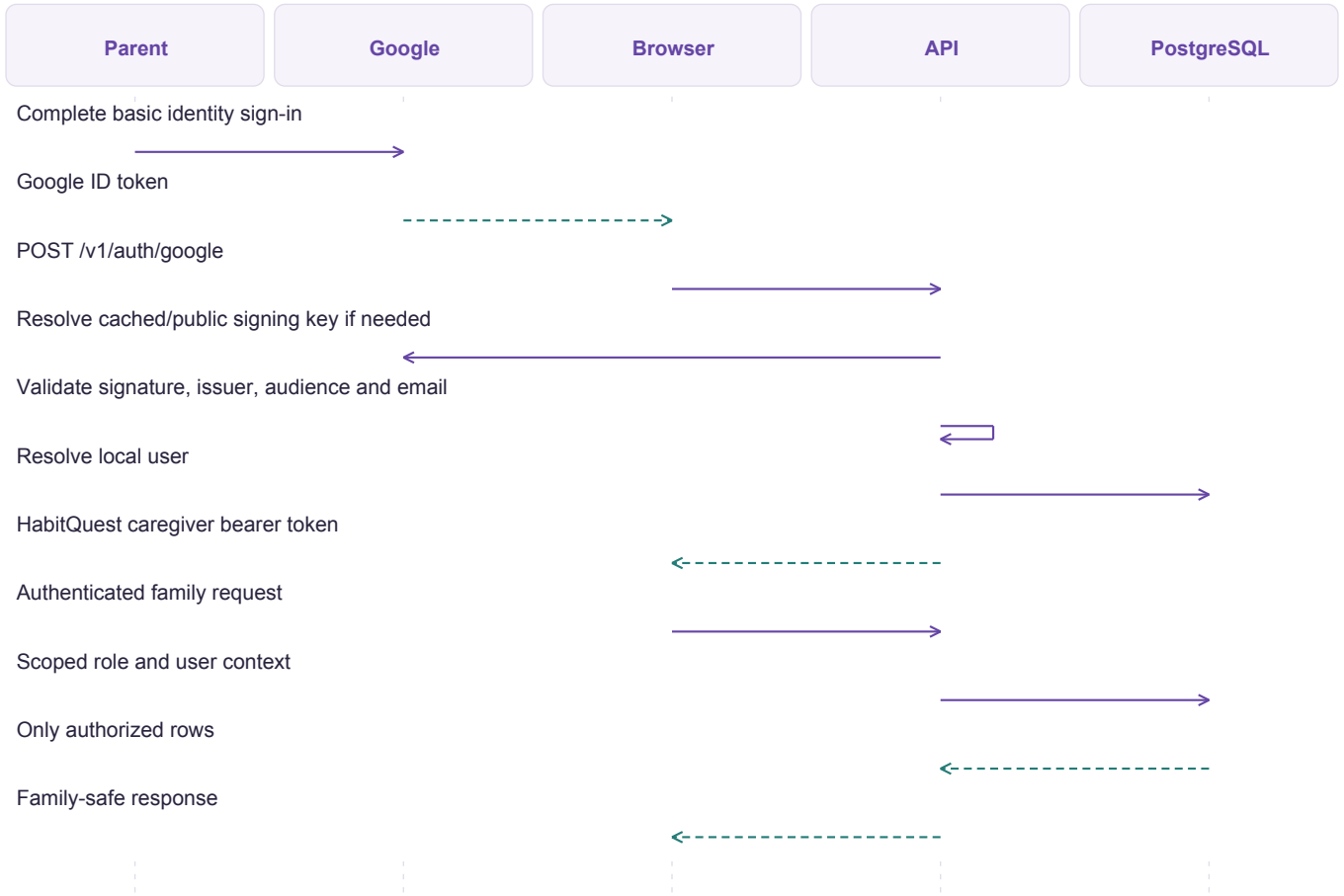
### Kids

A caregiver can request a token for one child when handing over a device. A child can alternatively use the family login code and their own PIN on another device. The token's role is `kid`, and its subject is that child's ID. Kid endpoints derive the child from the token; they do not trust a user-supplied sibling ID.

Caregiver and kid credentials are rejected on each other's endpoint families. Frontend route guards improve the experience, but backend identity checks and database policies enforce the boundary. The optional family PIN provides an additional grown-up confirmation for relevant actions and shared-device exit. It is distinct from the child's own sign-in PIN and from operator credentials.

### Operators

`/admin` is an independent browser entry and `/v1/admin` is an independent API boundary. Operator tokens have a separate signing secret, audience, issuer suffix, role and revocation version. Their configured default lifetime is 30 minutes. The browser keeps the admin token in memory. An operator is not added to a family and does not inherit the caregiver API's data access.




Sources: authentication, account/session endpoints, operator authentication, browser admin entry.

## Database authorization and atomicity

Ordinary domain operations connect with psycopg, begin a transaction, set the local database role and set transaction-local identity variables. A successful context commits; an exception rolls back. The shared helper sets a three-second connect timeout and ten-second statement timeout.

| Database role | Scope |
|---|---|
| `hq_api` | Caregiver domain operations scoped by `app.user_id` and family membership |
| `hq_kid` | Narrow reads/writes for `app.child_id`; no general child-profile or parent-content access |
| `hq_meter` | Internal operation/attempt accounting and validated family/child scope lookup |
| `hq_admin` | Tokenomics policy, pricing and sanitized accounting views; not general family content |

The deployed runtime login is a restricted `NOINHERIT` role allowed to enter the needed roles explicitly. It is not the RDS master or migration operator. RLS provides defense against missing family predicates, while SQL constraints, triggers, row locks and narrow security-definer functions protect cross-row invariants. Public reference data is intentionally different from private family tables; “every database row is private” would be an inaccurate description.

Examples of invariants include:

- Another family's inaccessible record returns 404 without confirming it exists.
- A generated habit has a durable owning child; it cannot become a sibling's ordinary custom habit if a draft/detail row disappears.
- Accepted generated instructions cannot be edited through custom-habit routes.
- A kid receives only the child-facing projection of a generated habit.

- Draft acceptance locks the relevant child/batch and rechecks PIN, age, reset fence, expiry, provenance and plan capacity before the atomic write.
- Buddy selection validates the final age and expected revision while holding the child lock, preventing a parent/kid race from silently overwriting a choice.

Sources: transaction helper, generated habit ownership migration, draft acceptance, buddy validation, Tokenomics grants.

## Transport, retries and operational boundaries

- `/health/live` establishes process liveness; `/health/ready` performs a small database read. Neither performs a paid model request.
- Optional SSE sends fixed stage identifiers, elapsed time and the final JSON result. It does not expose unreviewed model drafts. Provider work already dispatched or a database write already committed cannot be undone by closing the browser connection.
- The browser does not automatically replay failed writes. Explicit generation IDs, fingerprints and persisted results bound retries and accounting.
- CORS permits the configured frontend origin. CORS is not authentication; the API independently verifies bearer tokens and database access.
- Sign-in and PIN rate limiting is currently process-local. It resets on restart and is not a distributed abuse-control service.
- Behind the ALB, the API trusts only the last ALB-appended address when the actual socket peer is in the configured proxy subnets. Uvicorn runs with `--no-proxy-headers`; arbitrary caller headers are not accepted as identity.
- Startup does not run migrations or seed the deployed database. Those are deliberate operator activities with a named target and preserved migration ledger.

Sources: progress transport, API client retry boundary, address and attempt limits, deployment runtime.

## Security design process and validation

The design combines visible parent decisions with backend authority and database constraints. Later source-goal, generated-habit, Tokenomics and buddy capabilities were additive; they retain existing family ownership, accepted content and progress. Each extension has tests for wrong-role access, cross-family/child access, stale requests and replay, rather than relying on UI filtering alone.

These are implemented boundaries, not a claim of a completed independent security audit. Current limitations include browser-stored family bearer tokens, in-memory attempt limits and a single prototype runtime deployment. Any scaling or identity redesign must preserve or strengthen the existing contracts.

Test evidence: accounts, admin, proxy trust, generated habits, habit audience contract, buddy authorization/races.

# 04 Database and domain model

PostgreSQL is the authoritative store for HabitQuest accounts, child plans, accepted habits, daily progress and generation accounting. Pinecone is a semantic retrieval index over the researched public catalog, not the family database. This document reflects the 22 SQL migrations present on November 6, 2026: they define 36 tables in `hq`, plus the separate `public.hq_migrations` ledger.

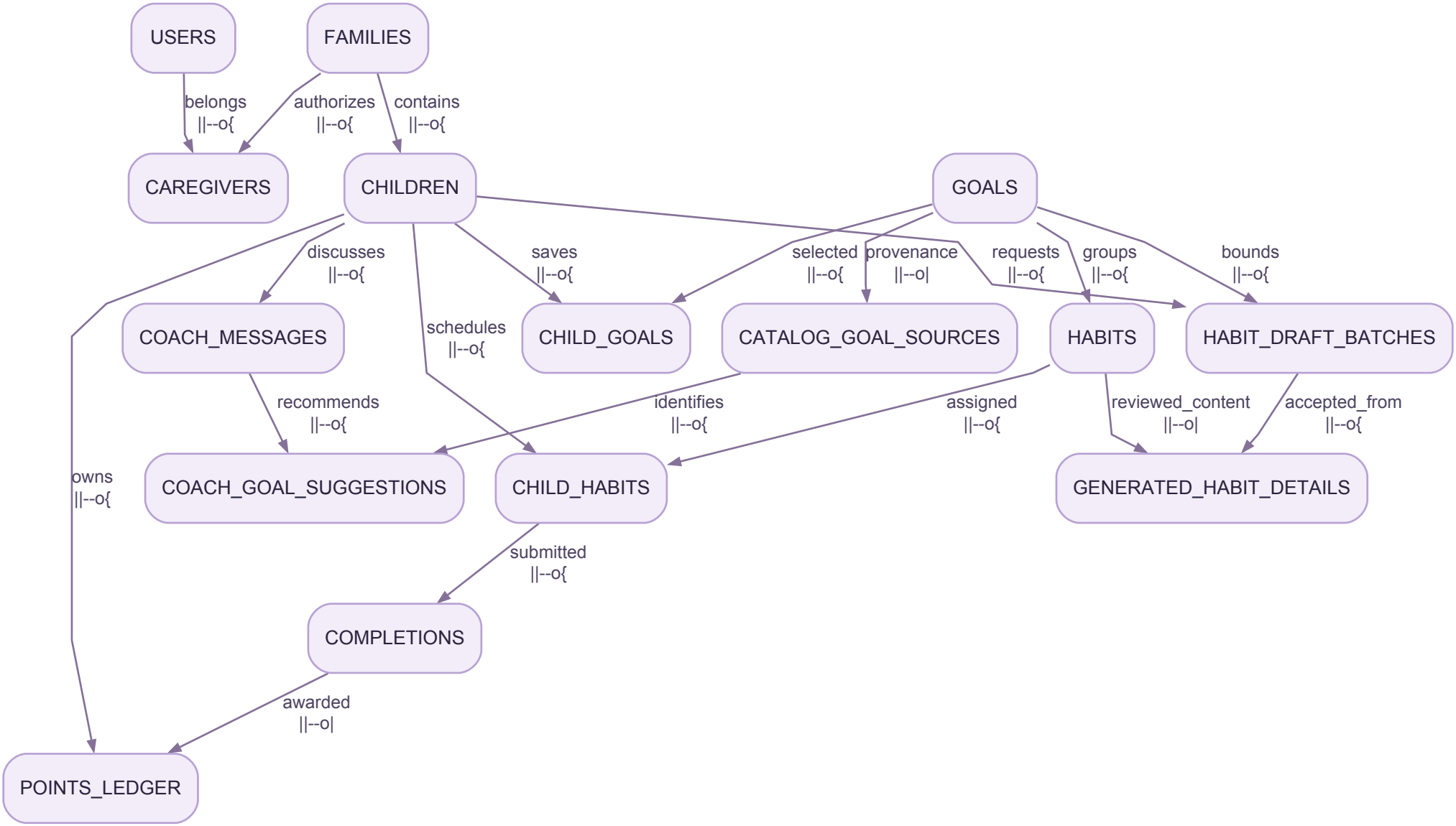
## Storage boundaries

| Store | What belongs there | What it does not decide |
|---|---|---|
| PostgreSQL `hq` | Family ownership, profiles, plans, accepted content, completions, points, reports and accounting | Semantic similarity ranking by itself |
| Packaged canonical catalog | Pinned source IDs, cards, hashes and provenance used to validate retrieved hits | Which goals are already saved for a child |
| Pinecone | Searchable catalog vectors and metadata | Child assignments, family data or points |
| Frontend presentation manifests/assets | Public goal/category art and buddy metadata | Whether a goal exists in the application database or is assigned |
| Browser storage/cache | Session context, UI preferences, snapshots and optional local timing history | Authoritative approvals, balances or assignment ownership |

The app can package presentation for 381 source cards without importing all of them into PostgreSQL. A source card becomes an `hq.goals` row only through the validated generation/materialization path. Browsing source detail and rendering an image are read-only activities.

Sources: catalog repository, lazy materialization migration, retrieval, presentation component.

## Core entities and relationships




Cardinality follows the source diagram: || means exactly one; o| means zero or one; o{ means zero or many.

The diagram shows important foreign-key relationships, not every column or table. The base schema reference remains useful for original tables, but the migration files and this design include later source-goal, generated-habit, Tokenomics and buddy additions.

## Goal, habit, draft and assignment are distinct

| Concept | Main record | Meaning and lifetime |
|---|---|---|
| Goal | `goals` | Desired skill/outcome; global catalog identity |
| Canonical binding | `catalog_goal_sources` | Source card/concept/category/version/hash/snapshot tied to a materialized goal |
| Child goal | `child_goals` | This child's suggested, active, done or dropped goal |
| Coach recommendation | `coach_goal_suggestions` | Ordered, pending/dismissed source-goal suggestion owned by the child's conversation |
| Draft batch | `habit_draft_batches` | Private child/goal-bound generation, request identity, source snapshot and choices; default expiry 24 hours |
| Habit | `habits` | Reusable catalog/custom content or an accepted private generated action beneath exactly one goal |
| Accepted generated detail | `generated_habit_details` | Reviewed structured instructions, source, batch/draft identity and content digest |
| Assignment | `child_habits` | A child's schedule for a habit: cue time, days, active/paused/removed status and dates |

For imported planning content, **the researched habit card maps to a goal**, and **the generated action maps to an app habit**. Category similarity is never a substitute for the `goal_id` foreign key. `my-ai-` generated habits retain `generated_child_id`; the database enforces that they cannot be assigned to a sibling or rewritten through ordinary custom-habit updates.

Source materialization checks the caller's child and final age, permitted setting, eligible intent class, language, release version and canonical SHA-256. A pinned allowlist and advisory lock prevent untrusted source insertion and conflicting concurrent materialization. An existing incompatible binding fails rather than silently rewriting the catalog identity.

The plan limit counts six non-removed assignments, including paused ones. Drafts do not consume capacity. Explicit acceptance checks room under a child lock; removal preserves assignment/completion history and makes capacity available. Restoration reuses approved content. Chat reset cancels/invalidates conversation work while preserving accepted plan content and its identity.

Sources: catalog binding migration, draft/detail schema, ownership protection, plan operations, draft lifecycle.

## Daily completions and two different reward measures

`completions` has one row per assignment and day. Submission, approval and rejection are explicit states. The child's Today endpoint calculates the day in the family's timezone and projects only relevant scheduled assignments. The full Plan endpoint returns the broader assignment set, including habits scheduled on another day.

Points are the sum of `points_ledger`, with a unique completion reference for an earned award. Approval and ledger updates are transactional; duplicate approval does not create another payment. Redemption/reward operations record negative ledger entries and enforce the available balance. Reward requests also represent their pending/decided state, including reserved spending behavior.

Sparks are separate progress. Current journey code sums approved habit tiers per day, capped at six daily completion Sparks, and adds stored journey bonuses. Pending Sparks are calculated separately. World thresholds are 0, 30, 90, 180, 300 and 450; quest stars use approved-completion thresholds of 5, 15 and 30. The old base-migration comment saying simply “count of approved completions” is historical and does not describe the final journey calculation.

Sources: daily loop, journey calculations, points/rewards, daily-loop schema.

## Table inventory by capability

| Area | Tables |
|---|---|
| Identity and profiles | `users`, `families`, `caregivers`, `children`, `family_settings` |
| Catalog and plans | `goals`, `habits`, `pain_points`, `child_goals`, `child_habits`, `catalog_goal_sources` |
| Conversations and generation | `coach_messages`, `ai_suggestions`, `coach_goal_suggestions`, `ai_usage`, `ai_reports`, `habit_draft_batches`, `generated_habit_details` |
| Daily loop and family features | `completions`, `points_ledger`, `rewards`, `reward_requests`, `checkins`, `surprises`, `surprise_reveals`, `child_extras` |
| Public buddy reference | `buddy_catalog`; selection mode/ID/version/revision are additive columns on `children` |
| Current Tokenomics | `tokenomics_policy`, `tokenomics_policy_audit`, `ai_price_versions`, `tokenomics_exposure`, `family_ai_windows`, `ai_operations`, `ai_provider_attempts` |
| Preserved earlier budget records | `planning_budget`, `planning_reservations`; historical carryover, not current app admission authority |
| Migration integrity | `public.hq_migrations`, outside the `hq` schema |

Tokenomics stores policy/price snapshots, invocation/attempt usage, timings, estimated cost and reservation exposure independently from displayed family credits. Parent/child operation links bind automatic habit slots to the Coach request. It preserves prior exposure across the accounting cutover. See Tokenomics design.

## Concurrency, access and replay

- RLS scopes family tables to caregiver membership or the authenticated child. Public reference reads and internal metering/admin roles have separate policies.
- `(child_id, request_id)` makes draft requests idempotent; the input digest rejects a reused key with changed input.
- Draft completion/acceptance rechecks child age, reset marker, expiry and source. A late result cannot resurrect choices after reset or an incompatible age edit.
- `(batch_id, draft_id)` binds one accepted detail record to a reviewed choice.
- A partial unique assignment index prevents duplicate non-removed assignments for the same child and habit.
- `(child_habit_id, day)` and the unique points completion reference protect the daily loop against duplicate submissions/awards.
- Buddy selection uses an expected revision and row lock; ordinary old-client profile writes cannot overwrite a deliberate newer selection.
- AI operation request keys, fingerprints and bound goal slots prevent duplicate metering and unbounded automatic preparation.

These constraints complement API validation. They are not delegated to the LLM. See API and access boundaries.

## Migrations, development and recovery

`scripts/setup_db.py` explicitly selects `DATABASE_URL`, obtains an advisory lock, applies migrations in filename order and records each file's SHA-256. A changed already-applied file raises an error. The catalog seed runs only when that operator script is invoked; the current AWS container does not automatically migrate or seed on startup.

The earliest migration contains a historical destructive schema reset. Therefore an imported database must retain its migration ledger: restoring rows without their ledger and replaying every migration is not a valid deployment approach. New changes require new migrations and a tested backup/restore plan. Tests that empty tables use disposable databases ending in `_test`, never the real app DB.

AWS uses always-on PostgreSQL RDS for this demo; the local app keeps its separate local PostgreSQL. The deployment record contains the existing backup/import and connection procedures. This documentation review performs no live schema/data changes and does not claim a fresh production table audit.

Sources: migration runner, migration files, schema tests, plan-management tests, cross-surface planning journey, RDS import record.

# 05 AI orchestration and safety boundaries

Current implementation reviewed November 6, 2026. This describes the deployed `COACH_ENGINE=refined` path. The earlier single tool-calling agent and crew remain in source as selectable alternatives; their diagrams are not the active design.

## Design in one paragraph

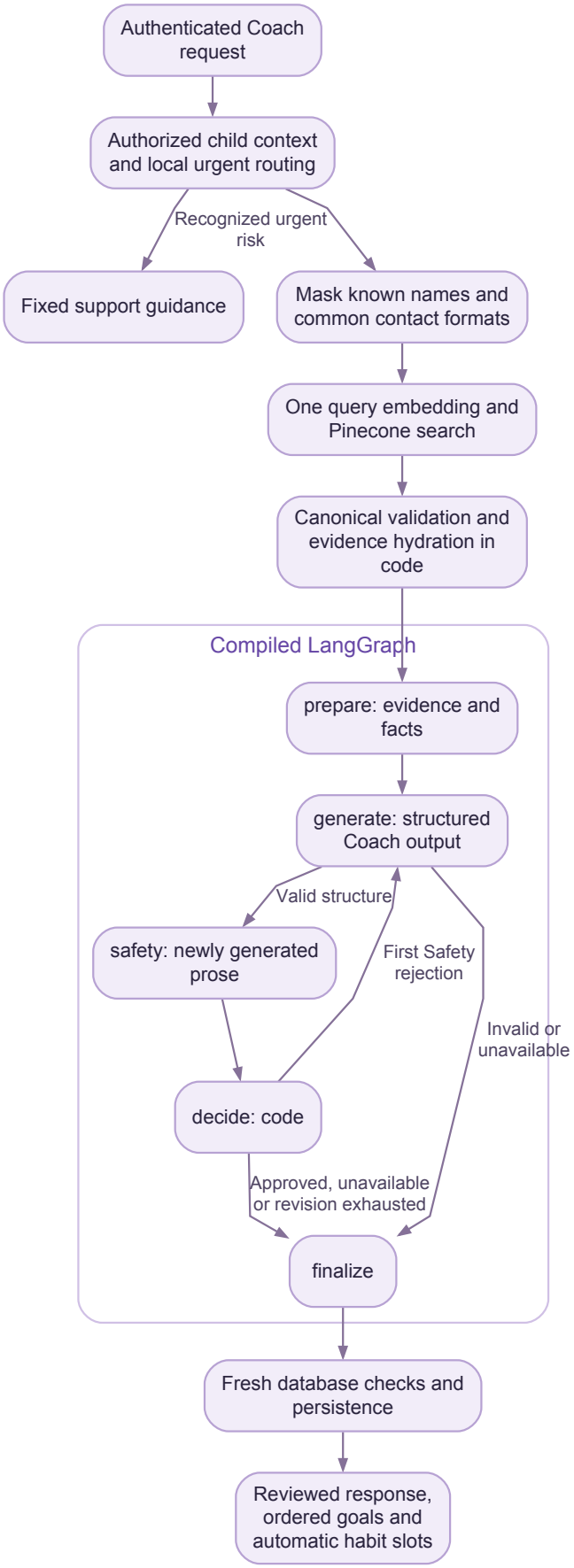
HabitQuest uses LangChain for provider clients and typed model output, and LangGraph for a small, explicit Coach state machine. Code retrieves source cards, computes progress facts and enforces authority. A central generator writes advice and selects allowed goals. A separate model reviews the newly generated prose. There is one possible revision, followed by a final decision in code. Habit writing, weekly review and Quest Planner are separate operations, rather than hidden agents invoked by every chat message.

## What “agentic” means here

There are **five nodes in the active Coach graph and two model-backed roles**. Five graph nodes do not mean five LLM calls. Retrieval is prepared before the graph invocation and is not a generative “research agent.”

| Active graph node | Responsibility | Chat-model calls |
|---|---|---|
| `prepare` | Assemble bounded evidence, age, conversation, plan and deterministic progress facts; construct allowed choices | 0 |
| `generate` | Produce structured advice, goal selections and proposed changes from the evidence | 1 per pass |
| `safety` | Review newly generated conversational text, questions and explanations | 1 per pass |
| `decide` | Permit a single revision only after a Safety rejection | 0 |
| `finalize` | Publish reviewed output or a fixed failure response with an explicit outcome | 0 |

The graph is compiled once. Per-request runtime context supplies the authorized snapshot, settings and result accumulator; requests do not share mutable family state through the graph definition.




Preparation failure also goes directly to `finalize`. Graph nodes check the same monotonic deadline before starting more work or publishing late output.

## Call budget by operation

“Logical call” means one application invocation of a model. An SDK retry is a separate HTTP attempt, even if it belongs to that same logical call.

| Operation | Normal logical chat calls | Allowed additional work |
|---|---|---|
| Refined Coach | 2: generator + output Safety | One rejected-text revision: another generator + Safety, 4 total |
| Query retrieval | 0 | One embedding request and one Pinecone Documents search; no chat summarizer or model safety |
| Habit batch | 1 writer | One correction for failed local validation, 2 total |
| Quest Planner | 1 | SDK retry policy only; no Coach graph or retrieval |
| Weekly review | 1 | SDK retry policy only; no Coach graph or retrieval |
| Legacy buddy line | Up to 2 | One generator and one child-facing screen, with fixed fallback |
| Catalog buddy line | 0 | Fixed event copy |
| Child-written note screen | Up to 1 | Local checks run first |
| Onboarding goal mapping | Up to 1 | Local risk routing and keyword/manual fallback |

A Send returning four new goals can subsequently trigger four separate writer batches. Its normal total is therefore **six chat calls across five operations**, plus the embedding and search, even though the Coach graph itself needs two. Those writer requests are explicit preparation authorized by the Send action, limited to two in flight. There is no automatic assignment to a child's plan.

See Coach and habit generation for that parent interaction and other AI capabilities for the independent report and child-text workflows.

## Safety and authority are different checks

| Boundary | Implemented protection | What it does not claim |
|---|---|---|
| Before external retrieval | Fast code masks known child/caregiver names and common email/phone formats | Comprehensive PII recognition or a model-based input review |
| Local urgent routing | Recognized risk phrases return fixed support guidance before inference | Clinical assessment or exhaustive understanding of unusual wording |
| Pinecone results | Pinned release, source IDs, metadata/hash, age/context and concept checks | Another safety review of the already curated catalog |
| Coach structured output | Dynamic allowed-ID schema, relationship/eligibility bounds and local validation | That model-selected relevance has been independently proven |
| Coach prose | Separate generated-text Safety verdict; any problems reject approval | Review of every canonical source field as if newly authored |
| Habit drafts | Local schema, source, age, wording, duration, privacy and action-novelty checks; parent preview | A separate model safety verdict on the entire habit |

| Boundary | Implemented protection | What it does not claim |
|---|---|---|
| Acceptance | Current authorization, optional parent PIN, ownership, goal linkage, eligibility and plan-capacity checks | Permission for AI to apply changes on its own |

The `selected_goal_context` supplied to Safety helps interpret the generated explanations. The catalog is trusted context, not new text being approved or rewritten. The old remote `ParentRisk` step is not called by refined Coach or habit writing. Legacy engines and child-text screening retain their own rules.

## Deterministic work replacing the former crew

- **Triage:** the explicit UI operation and central Coach interpret the request; there is no separate triage model.
- **Research:** retrieval returns curated, versioned source records. Their research support and limitations are already attached; no live browsing or research summary model runs during a chat.
- **Patterns:** SQL/context code supplies counts and plan facts. Weekly review is the separate, parent-initiated place for narrative interpretation of progress.
- **Fit:** code constrains allowed IDs, relationships, age, context and capacity; the generator must ground its explanation in the candidate. There is no remote Fit model or fit-driven revision loop in refined mode.
- **Decision:** code decides whether to revise, publish or return a fixed failure.

The original planning-service prototype uses an actual Deep Agents harness with bounded tools and checkpoints. HabitQuest adapted its catalog, retrieval and action-generation capability into backend modules. It does **not** make a nested HTTP call to that prototype or run its multi-invocation harness inside the active Coach graph.

## Failure, concurrency and observability

Provider failure, malformed output, timeout or exhausted Safety revision returns a named outcome and retains earlier usable recommendations. A retrieval failure is not presented as proof that no matching catalog card exists. Raw unreviewed model text is not streamed to the browser; progress events report actual stages, followed by the final validated result.

The configured Coach outer deadline is 65 seconds. It prevents subsequent dispatch and late persistence but cannot retract an HTTP request already accepted by a provider. Habit writing has its own 45-second deadline. These are failure bounds, not expected response-time promises.

The endpoint metering layer tracks operation identity, reservations, attempts, tokens, cost estimates and settlement separately from graph state. Since the November 6 fix, reviewed final outcomes such as `recommendations` and `advice` are recognized as successful instead of being misclassified as zero-credit failures. That also preserves automatic habit slots. See Tokenomics and latency.

## Source of truth and verification

- Refined graph and prompts
- Request orchestration, privacy, retrieval and persistence
- Context cache and authoritative fingerprints
- Provider adapter, deadlines and attempt counting
- Outcome settlement
- Graph regression tests and Coach integration tests
- Historical reasoning: refinement operations and integration execution

The dated execution records contain live samples; mocked regression tests prove contracts and failure boundaries, not provider relevance or a production p95. The final design retains the five-second useful-response goal without claiming that every first response, revision or multi-goal preparation meets it.

# 06 Habit Coach, source goals and generated habits

Current design reviewed November 6, 2026. A canonical catalog **habit card becomes a HabitQuest goal**; a generated concrete action becomes a **habit beneath that goal**. This translation preserves the existing app's daily quests, scheduling, parent approvals, Sparks and points.

## The parent experience

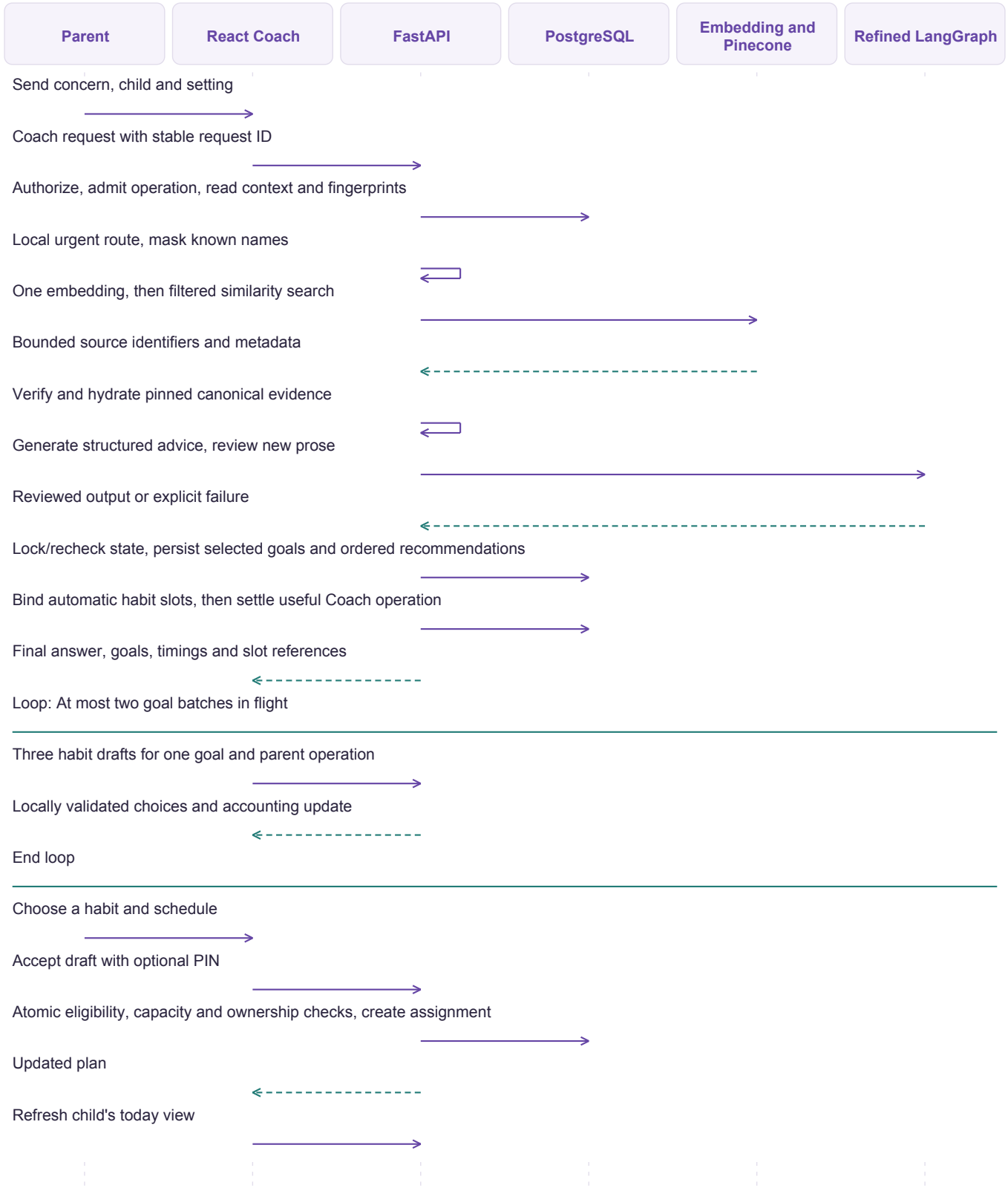
Coach is a child-scoped conversation. A compact child selector shows age, and the daily credit chip sits with New chat. Small location icons share the composer area. Code suggests a location from the concern or source context; the parent can override it. Location selection makes no model call.

The reviewed answer is displayed as brief guidance with ordered steps. New goals appear first, with clear separation from earlier recommendations and saved goals already in the plan. Cards use catalog artwork, age/context information and research summaries. Modal readers hold the fuller source and habit details.

- A new Send aims for three or four relevant goals, with a hard maximum of four; it does not fabricate goals merely to fill the screen.
- Three habit choices are automatically requested for each permitted new goal, with no more than two batches running at once.
- Existing usable saved batches are reused. Passive page loads and saved-draft reads do not spend or launch generation.
- **Get more goals** appends additional relevant goals while preserving prior choices. **More habit choices** explicitly requests another bounded batch of three; it does not discard the existing batch.
- Generated choices remain proposals. The parent selects the habit, schedule and optional PIN confirmation before it joins the child's plan.

The current Coach does not render inline “credits used,” preparation accounting or per-batch price copy beneath its answers/cards. The compact daily meter remains. Generation stages and optional seconds remain independent display features. An already-open production tab needs a reload after deployment; local Vite refreshes changed development code automatically.

## End-to-end request and persistence




The browser renders stage events while work is in progress. It waits for reviewed Coach output before showing generated prose. Habit batches continue separately after the goals arrive, so “time to Coach answer” and “time until every goal has three choices” are different measurements.

## Retrieval: what “deep search” does

The active source-goal path performs one masked query embedding using the pinned 512-dimensional embedding model and one Pinecone Documents similarity search. Hard filters include age, selected context and catalog release; previously shown concepts can be excluded for more-goals requests. The search asks for at most 12 matches and hydrates at most six distinct eligible concepts for the generator.

The API checks returned identity and metadata against its packaged catalog, including source provenance and embedding compatibility. It then uses the local canonical content for the model and source reader. Search result text is not allowed to override the canonical record. Retrieval neither writes to Pinecone nor creates all catalog goals in PostgreSQL.

Follow-up resolution is deterministic: a short refinement retains the recent explicit objective, while an identified topic change establishes a new objective. This is a bounded heuristic rather than a universal intent classifier. The full parent feedback still reaches the Coach generator.

The historical `G1`-style guidance citations belong to the older local-guidance mode. The current source-goal path uses the catalog's research references, support statements and limitations. It performs no live research browsing and does not assert that each generated action has independently demonstrated efficacy. See research and Pinecone catalog.

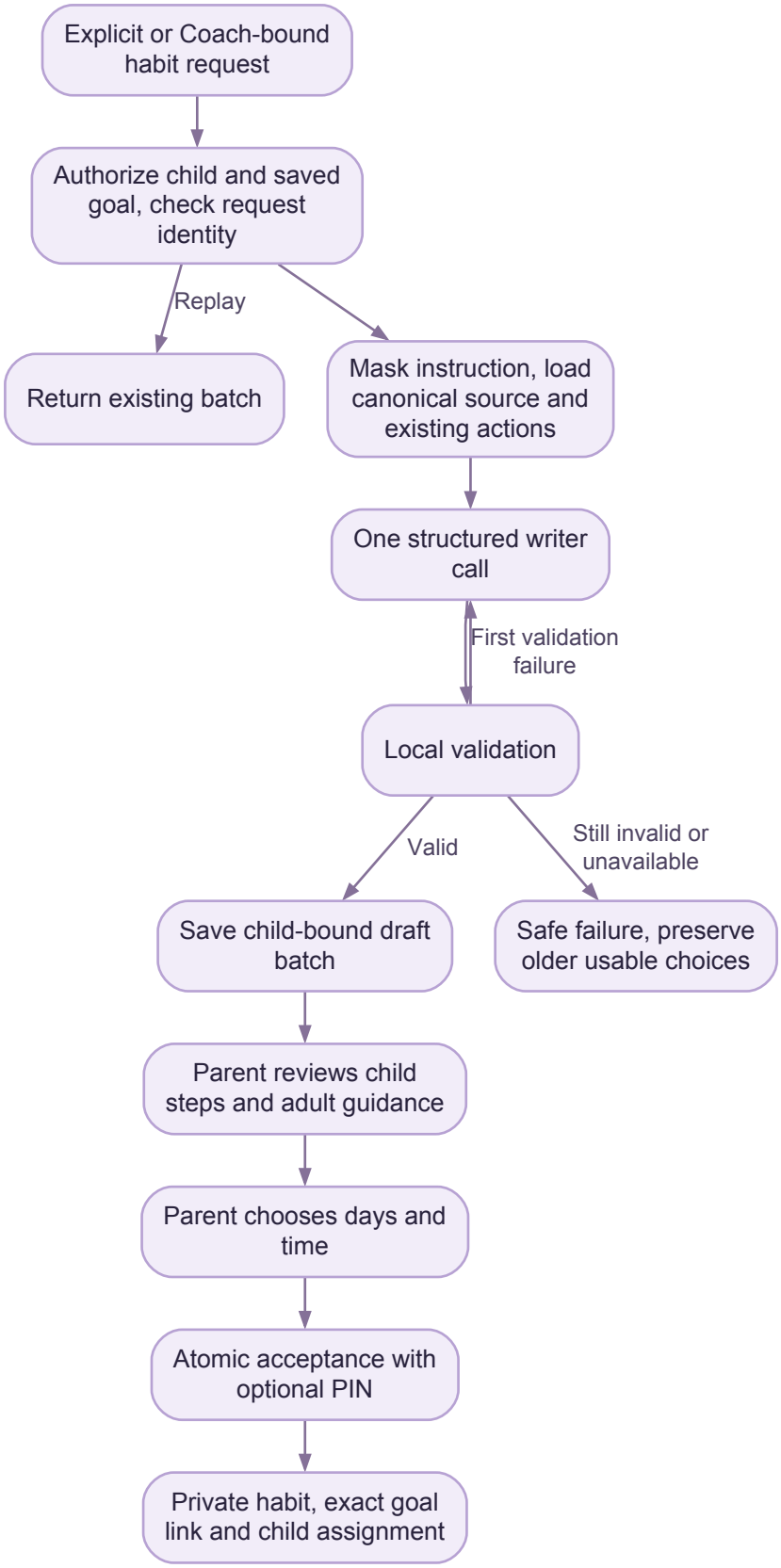
## Lazy goal materialization

A stable `hqg-...` identifier maps a specific canonical source record, including its age variant, to an application goal. Only goals selected by reviewed Coach generation are materialized with their source provenance. Reading a source card, retrieving candidates or obtaining a failed/rejected response creates no goal.

Ordering and lineage are explicit: recommendation order survives reload; canonical lock ordering prevents conflicting writes without reordering the UI. A saved goal recommended again appears with the latest response and a saved label, rather than becoming a duplicate goal. A habit links to its exact source goal; category similarity is not used to infer or repair that relationship.

## The separate habit writer

`POST /v1/children/{child_id}/goals/{goal_id}/habit-drafts` accepts a count from one to three; the current UI explicitly requests three. The goal must already be materialized and authorized. The writer consumes its canonical evidence, child age, setting, parent instruction and previously generated actions. It does not repeat retrieval or invoke Quest Planner.




The output separates short child instructions from adult preparation, explanation and coaching guidance. The schema includes readable steps, completion criteria, materials, a cue, optional help, an action-appropriate Fluent icon and a duration estimate. The estimate is not a deadline imposed on the child.

Validation checks exact count, shape, source grounding constraints, age-related wording, allowed duration, privacy leakage and novelty against previous actions. Only a local validation failure permits one correction call. No independent whole-habit Safety model is added. These checks plus parent approval are a bounded validation strategy, not a claim of exhaustive semantic safety review.

## Data and lifecycle safeguards

| Concern | Current rule |
|---|---|
| Duplicate generation | A stable request ID and input digest return the existing batch or reject incompatible reuse |
| Child/family isolation | Requests, drafts, provenance and accepted habits are authorized and child-bound |
| Reset or age change | Conversation/reset and age checks prevent accepting stale drafts |
| Plan capacity | Acceptance rechecks the six-active-habit limit under the child lock; removing a habit makes room |
| Goal linkage | Accepted `my-ai-...` habits retain the exact materialized source goal and lineage |
| Parent control | No automatic assignment; schedule and optional PIN are checked on acceptance |
| Existing accepted content | Approved generated instructions remain immutable through the ordinary editor |
| Child display | Kid APIs project child-facing fields; parent setup and adult coaching copy stay caregiver-only |
| Rewards | Existing tier-1/custom-habit 10-point policy applies; catalog-card reward rules are not imported |
| More choices | Append bounded batches, keep earlier choices and avoid automatic retry loops |

Daily recurrence uses all seven days; weekly recurrence uses selected days. Interval/as-needed schedules are not silently translated into supported schedules. An API plan update refreshes the dashboard/today projection so newly accepted habits appear for the correct child, including habits scheduled for other days.

## Credits and response timing

Coach and each habit batch have separate operation records. The usual configured weights are two credits for a useful Coach response and one for a useful batch; the Coach operation includes its embedding/search costs. Automatic batches carry the parent operation ID so reservations and totals remain connected. Provider retries and user request replays are not treated as new user operations.

The November 6 correction recognizes final successful graph outcomes `recommendations`, `advice`, `clarification`, `no_direct_fit` and `out_of_scope`. Previously the metering wrapper only recognized the older `reviewed` state and could zero the debit and cancel automatic slots after a successful response. The corrected success classification preserves preparation slots and refreshes the toolbar meter after Coach and each batch. Historical zero-credit operations are not backfilled or automatically replayed.

## Implementation references

- Coach frontend, goal cards and bounded queue, rich habit reader
- Coach API, retrieval, canonical repository
- Draft generation and acceptance, action quality, safe validation diagnostics
- Frontend queue/credit regression, API/source-goal regression, draft regression
- Deployed correction and browser refresh investigation

Related design: AI orchestration, database, Tokenomics and latency.

# 07 Quest Planner, weekly review and other AI capabilities

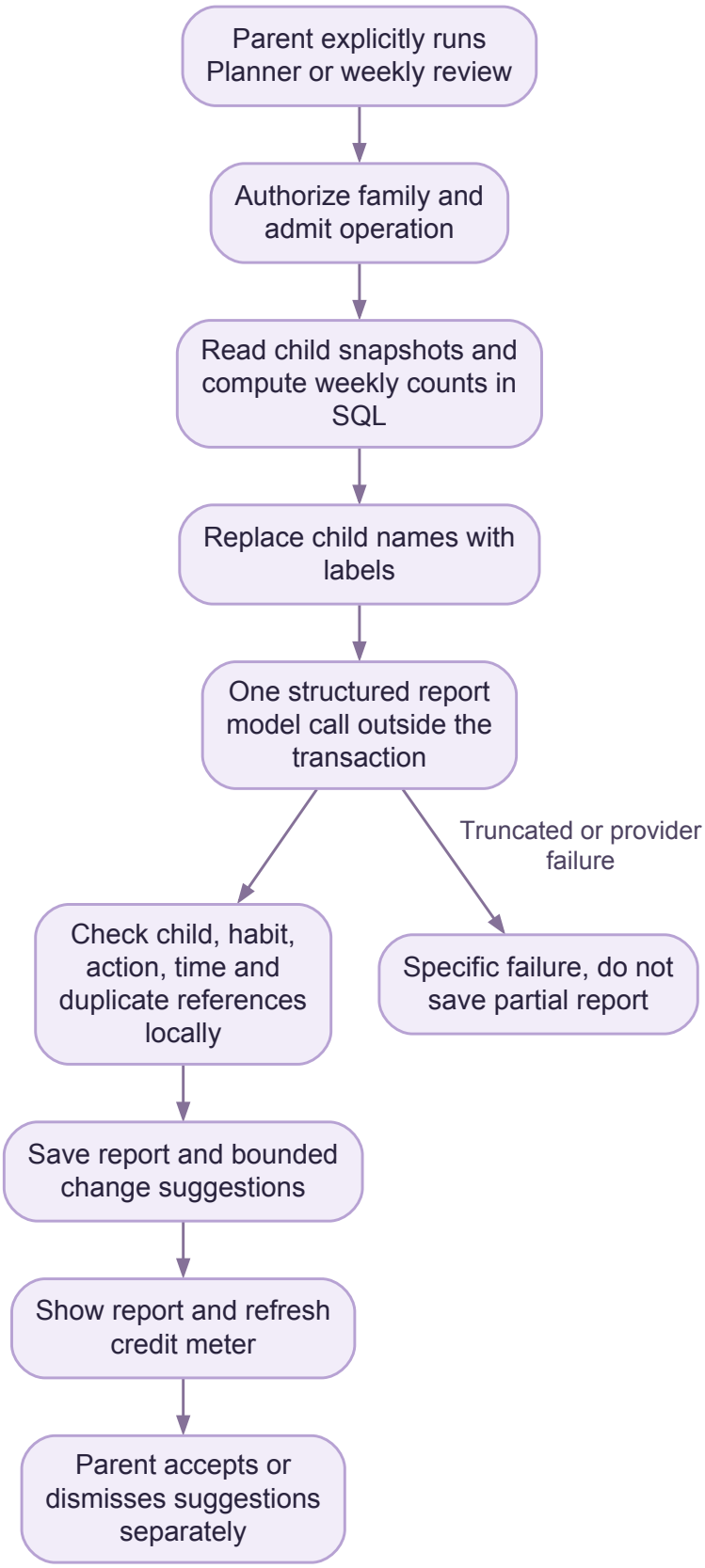
Current implementation reviewed November 6, 2026. These features are separate from the refined Habit Coach graph. No background scheduler periodically invokes them: the parent explicitly runs Planner or review, while child events invoke only their relevant text capability.

## Capability map

| Capability | Trigger and scope | Input | Model work | Stored/displayed result |
|---|---|---|---|---|
| Quest Planner | Parent dashboard; family | Each child's age, plan, recent approvals and check-in facts | One structured report call | Summary, practical tips and at most four checked change suggestions |
| Weekly review | Parent dashboard; family | Current seven days versus preceding seven days | One structured report call | Headline, per-child wins/watch points, patterns and at most two checked changes |
| Onboarding mapping | Parent describes a child's concern | Age, parent text and eligible catalog candidates | Up to one JSON mapping call | One to three suggested catalog goals and existing habit recommendations |
| Catalog buddy copy | Greet/submitted/celebrate for a child using a catalog buddy | Event | No model | Fixed event line |
| Legacy buddy copy | Greet/submitted/celebrate for a legacy buddy | Age, event and allowed quest titles | Generator plus child-facing screen | Short line or built-in fallback |
| Child note screening | Child submits a nonempty check-in note | Note and `kid-writes` audience | Local checks, then up to one classifier call | Save, friendly rejection, or risk flag visible to caregiver |
| Parent surprise text | Parent writes a message for the child | Message | Local checks only | Allowed message or rejection |

“Quest Planner” schedules and adjusts the existing family plan. It is distinct from the earlier **HQ AI services planning agent**, whose source-card retrieval and action-writing capability informed the Coach integration. Neither name means that ordinary chat silently runs the dashboard Planner.

## Report workflow shared by Planner and review




The model sees `Child A`, `Child B` and similar labels. The application restores names for display after generation. Context includes approved completion counts, plan information and structured check-in facts; it does not ask the model to count raw events. Custom habits are represented by their goal in the AI-facing snapshot instead of sending arbitrary private custom titles.

The provider call runs outside a database transaction. The later persistence transaction saves the report and valid proposals. Suggestions are not applied to the plan automatically; acceptance rechecks current plan rules through the existing suggestion workflow.

### Quest Planner

Endpoint: `POST /v1/families/{family_id}/plan-suggestions`.

The prompt asks for a realistic daily plan and modest adjustments, such as a 15-30 minute time change. It favors at most one new habit per child and a brief summary with two or three actionable tips. The code caps persisted proposals at four. Supported actions are add, retime, pause, resume and remove.

Code checks that the child label exists, an add refers to an available catalog habit outside that child's plan, other actions refer to the child's plan, retime values match the accepted time format, and duplicate child/habit pairs are discarded. Current eligibility and capacity are checked again on acceptance. Prompt constraints should not be confused with exhaustive formal validation of every narrative claim.

### Weekly review

Endpoint: `POST /v1/families/{family_id}/reviews`.

SQL supplies counts for the most recent seven days and the seven before them. The prompt asks for one win and one thing to watch per child, up to three evidence-based patterns and at most two proposed changes. Reports avoid sibling comparisons, shame and removal of earned rewards. A child with no habits should receive a short acknowledgement rather than invented progress.

`GET /v1/families/{family_id}/reports` reads the newest saved report of each kind without generating again. The frontend supports actual stage events and optional timing details for both operations.

## Report reliability and current review boundary

These report helpers use one structured model call plus local validation. **They do not invoke the Coach's separate generated-prose Safety model.** Their safety constraints are in the report prompts and the existing plan/acceptance rules. This is an implemented boundary, not an assertion that all AI output has the same independent review process.

An earlier “AI helpers are busy” failure was traced to output truncation: the configured reasoning model could use almost the entire 4,000-token allowance on reasoning before emitting its report. The current shared resolver applies `reasoning_effort=none` to Planner/review when `AI_HELPER_REASONING_EFFORT=auto` and the endpoint/model match the known Nebius DeepSeek V4 family. Other routes retain their provider default unless explicitly configured.

A truncated report now receives a specific 502 explanation; other provider failures retain the 503 response. No partial report or suggestion is saved. Provider attempts can incur cost even when the user operation fails. A confirmed failure lets the next explicit click use a fresh request ID; an interrupted or still-running request keeps its existing identity to prevent duplicate work. There is no automatic new-generation retry loop.

## Onboarding is the older catalog-mapping path

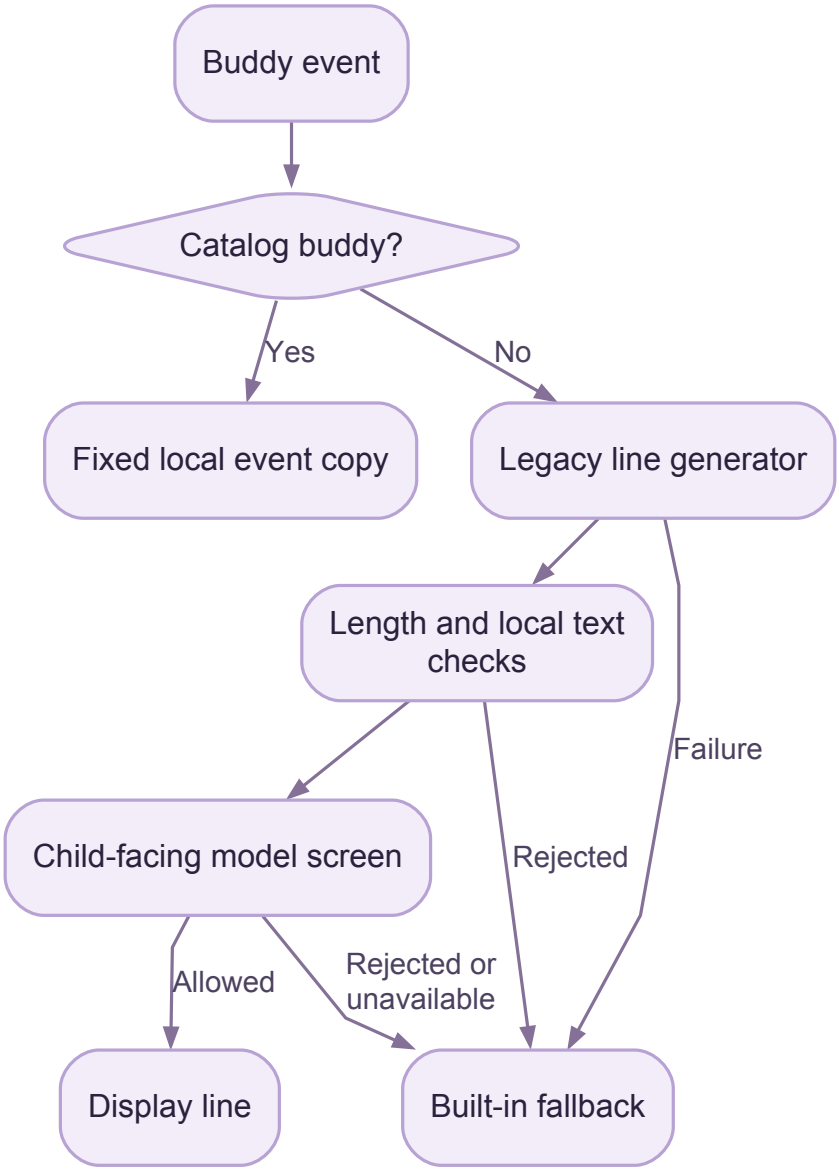
`POST /v1/children/{child_id}/pain-points` uses the existing application catalog, age and unlocked tier. It is not the Pinecone source-goal workflow. Local urgent risk phrases short-circuit generation; the model can also return an at-risk result. Structured mapping is checked against supplied goal/habit candidates. If inference fails, deterministic keyword matching or manual selection remains.

This path sends the parent-entered text with the child's age and candidates. It does not currently call the refined Coach's known-name masking helper. Therefore the refined path's stronger masking description must not be advertised as a universal property of every historical AI endpoint. No profile name field is added by this mapper, but a name typed in the parent's free text is still part of that text. This is a documented boundary for future privacy review.

## Buddy behavior and child-text checks

Catalog buddy selection, age-based recommendation, artwork, characteristics, outfit badges and event lines are local deterministic features. They do not need LLMs, embeddings or Pinecone. See buddies, artwork and assets.

The retained legacy buddy path asks the configured small/main model for a short line, normalizes it and rejects empty or overly long output. It then screens that line for `kid-reads`. Failure at any stage returns a built-in event line.




Child-note screening uses different failure semantics:

- Local patterns run first, with risk signals evaluated before ordinary blocking categories. A locally classified note needs no model call.
- If local checks pass, the configured child-screen model evaluates the note.
- Ordinary disallowed content returns a friendly correction message.
- Risk-bearing disclosures are retained and flagged for the caregiver rather than silently discarded.
- If the classifier is unavailable, `kid-writes` falls back to the local result so a child can still send a note. `kid-reads` fails closed and uses fixed copy.

The local patterns and model classifier are product safeguards, not a clinical assessment or guaranteed detection of every concerning message.

## Source and verification links

- Reports, buddy routes and prompts
- Model-routing resolver
- Onboarding endpoint and mapping logic

- Child text safety and check-ins/surprises
- Dashboard helper UI
- Report regressions, model adapter tests, helper UI regressions
- Historical diagnosis and evidence: report reliability repair

The report repair was verified with mocked provider responses and actual outgoing configuration assertions. Its historical latency examples are not new live benchmarks or evidence of a current production latency distribution.

# 08 Model selection and runtime routing

Current configuration reviewed November 6, 2026. This document records why the roles were separated, the model IDs actually configured in the deployed app and what evidence supports the choice. It is not a newly run provider comparison or a claim that the selected models are universally optimal.

## The decision

The primary improvement was to remove unnecessary model stages before attempting to make the old multi-call crew faster by swapping models. Retrieval, progress counts, catalog authority and final decisions now use code. Coach has one central generation plus one review of generated prose. Habit writing is its own bounded operation, and Planner/review run only when requested.

The working HQ planning prototype supplied the generation and query-embedding configuration. Its `gpt-6-luna` generation route was carried into Coach and habit writing with separate credentials, Responses API mode and low reasoning effort. The existing Nebius route was retained for output Safety and unrelated features to avoid changing all behavior at once. The original Deep Agents tool loop was not carried into the main Coach graph.

The literal model IDs below come from checked configuration and code. “Luna” refers to that configured API model ID. The earlier handoff's “scotch model” was interpreted as the Coach model, not a distinct API identifier.

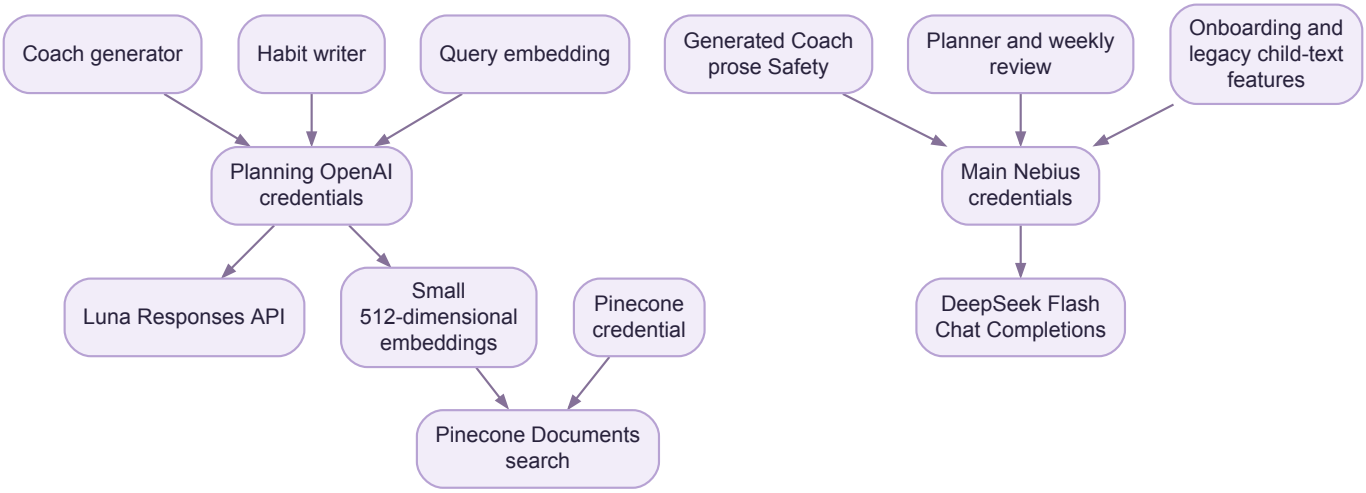
## Current role-to-model map

Source: deployed non-secret configuration, the settings schema and the same routing resolver used by invocation and admin metrics. The November 6 Coach release changed images, not this model configuration.

| Role | Exact configured model | Provider and API | Reasoning / normal work |
|---|---|---|---|
| Refined Coach generator | `gpt-6-luna` | OpenAI planning group; Responses API | `low`; one generation per pass |
| Habit writer | `gpt-6-luna` | OpenAI planning group; Responses API | `low`; one writer per batch |
| Generated Coach prose review | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; Chat Completions | `low`; separate output Safety call |
| Query embedding | `text-embedding-3-small` | OpenAI planning group; embeddings API | 512 dimensions; one masked query |
| Pinecone similarity lookup | `documents-search` accounting label | Pinecone Documents API `2026-07` | Vector search, not a text-generation model |
| Quest Planner | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; Chat Completions | `auto` resolves to `none` for this role/provider/model |
| Weekly review | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; Chat Completions | Same report-specific `none` resolution |
| Onboarding mapping | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; JSON-object output | Existing provider default reasoning behavior |
| Legacy buddy generator | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; Chat Completions | Small-model selector falls back to primary because its override is empty |
| Child-text screen | `deepseek-ai/DeepSeek-V4.1-Flash-1017` | Nebius main group; structured verdict | Same small-model fallback; not the dedicated output-Safety selector |

| Role | Exact configured model | Provider and API | Reasoning / normal work |
|---|---|---|---|
| ParentRisk | Same DeepSeek ID is configured | Main group; retained legacy capability | **Not called by refined Coach or the habit writer** |

Catalog buddies do not appear as a paid generation role because their current selection and event copy use no model.




## Configuration boundaries and limits

| Concern | Effective setting or rule |
|---|---|
| Main provider | `OPENAI_BASE_URL=https://api.tokenfactory.us-north1.nebius.com/v1/`; `OPENAI_API_KEY` is private |
| Imported planning provider | `PLANNING_OPENAI_BASE_URL=https://api.openai.com/v1`; independent private `PLANNING_OPENAI_API_KEY` |
| Coach routing | `COACH_PROVIDER=planning`, `COACH_MODEL=gpt-6-luna`, `COACH_API_MODE=responses` |
| Coach output allowance | 1,600 tokens; 30-second per-request limit and 65-second outer turn deadline |
| Coach SDK retries | One retry per logical call; a Safety revision is counted separately |
| Writer output allowance | `max(2800, 800 + 1100 x count)`; three drafts permit 4,100 tokens |
| Writer deadline/retries | 45-second operation deadline, zero SDK retries; one local-validation correction remains allowed |
| Output Safety allowance | Main adapter's 4,000-token limit, up to 30 seconds per request, one SDK retry |
| Planner/review | Main adapter's 4,000-token limit and one SDK retry; role-specific reasoning resolver prevents default thinking from consuming short-report output |
| Query embedding | Small/512, 15-second request limit, zero SDK retries |
| Pinecone | 15-second request limit, no retry loop; pinned index, namespace, field, metric and catalog release |
| Provider storage | Planning Responses clients explicitly set `store=False` |
| Startup validation | Missing enabled-feature credentials/model names, invalid declared API modes, incompatible provider/engine selection or wrong pinned embedding/index settings fail configuration checks |

Transport limits are further bounded by the remaining operation deadline. A timeout cannot recall a request that a provider already accepted. Tokens and HTTP attempts therefore remain visible in accounting even if no useful final result is delivered.

Configuration validation does not establish that a newly chosen provider/model supports every schema or parameter; that requires its own compatibility checks.

Actual keys are never in these documents, the frontend, build arguments or image layers. Local values belong in ignored backend `.env`; AWS injects secret fields from Secrets Manager into the API task. The browser has no provider credentials. Use backend/.env.example for commented configuration instructions and AWS deployment for the delivery boundary.

The bundled canonical release requires small/512 compatibility. Changing the query embedding model or dimensions without re-embedding the corpus and updating the reviewed release is rejected. The evaluation and release decision are covered in embedding-model selection.

## How the choice evolved

1. **Audit the actual calls.** The earlier crew separately called triage, research, progress, planning, fit and safety, with another planning pass after review. Investigations found that provider work and repeated generation could dominate the reported 45-50 second experience.
2. **Consider role-appropriate alternatives.** The November 3 Nebius analysis shortlisted a non-thinking Qwen model for routine work, GPT-OSS for structured planning/review and several challengers. It explicitly treated these as evaluation candidates, not measured winners.
3. **Simplify the architecture.** The refined design replaces routine analysis stages with bounded retrieval and code. A separate generated-prose review is retained; the deployed workflow is not literally one total model request.
4. **Reuse the tested planning capability deliberately.** Carry over the prototype's Luna/Responses writer configuration and small/512 embedding space, while retaining the main provider for other features. Adapt capability and canonical data, rather than nesting the prototype service or harness.
5. **Validate the actual schemas and failure modes.** Test structured output, source IDs, local correction feedback, Safety revision limits, privacy, idempotency and real provider compatibility with bounded synthetic checks.
6. **Tune the operation, not only the model name.** Reduce model-facing evidence to useful fields, keep substantive source limitations, request shorter copy and reuse HTTP connections. For Planner/review, fix reasoning controls that caused output-limit errors without changing models.

The historical shortlist remains useful design rationale. Its prices, availability, crew-role assignments and local-BM25 description are dated evidence, not the final deployed configuration or current vendor promises. No comparison proved that Qwen/GPT-OSS should replace the configured models, and they were not silently activated.

## What the measurements establish

The approved November 4 synthetic sequence recorded useful Coach responses at **6.692 s initial**, **4.336 s follow-up** and **3.996 s after a topic switch**, and a **4.612 s** single habit-generation request. These were API/provider/database round trips with no revisions, not browser paint times, twelve-habit completion times or a production latency distribution. Later saved writer requests were reported at 7.5-8.4 seconds. A five-second target is therefore not a consistent observed guarantee.

The November 5 report diagnosis found examples where reasoning consumed most or all of a 4,000-token output allowance. The repair verified the exact outgoing reasoning settings and truncation/retry behavior with mocked transports; it did not run a new paid latency benchmark. The November 6 release likewise used mocked generation tests and live infrastructure readiness checks, not new paid model comparisons.

Any next model comparison should keep the canonical release and request set fixed, change one role at a time, and measure:

- Useful-result rate, directly relevant goals and valid schema/source linkage.
- Child-wording, privacy, rejection and correction/revision behavior.
- Initial and warmed end-to-end p50/p95, with browser time separated from server and provider stages.
- Actual input/output/reasoning usage, retries, embedding/search cost and total cost per useful interaction, rather than token price alone.
- Failure and timeout rates, including incomplete or unusable output.

No paid comparison, model switch or rate refresh is implied by writing this design.

## Evidence and maintained references

- Historical Nebius selection analysis
- Configuration migration rationale
- Refinement execution and live samples
- Dated latency evidence
- Planner/review failure analysis
- Prototype configuration
- Model adapter tests and configuration tests
- Tokenomics and current measurement strategy

# 09 Embedding model selection

Reviewed November 6, 2026 against the recorded experiment, ingestion receipts and current application code. This document explains the completed selection; it does not report a new benchmark or new provider-price check.

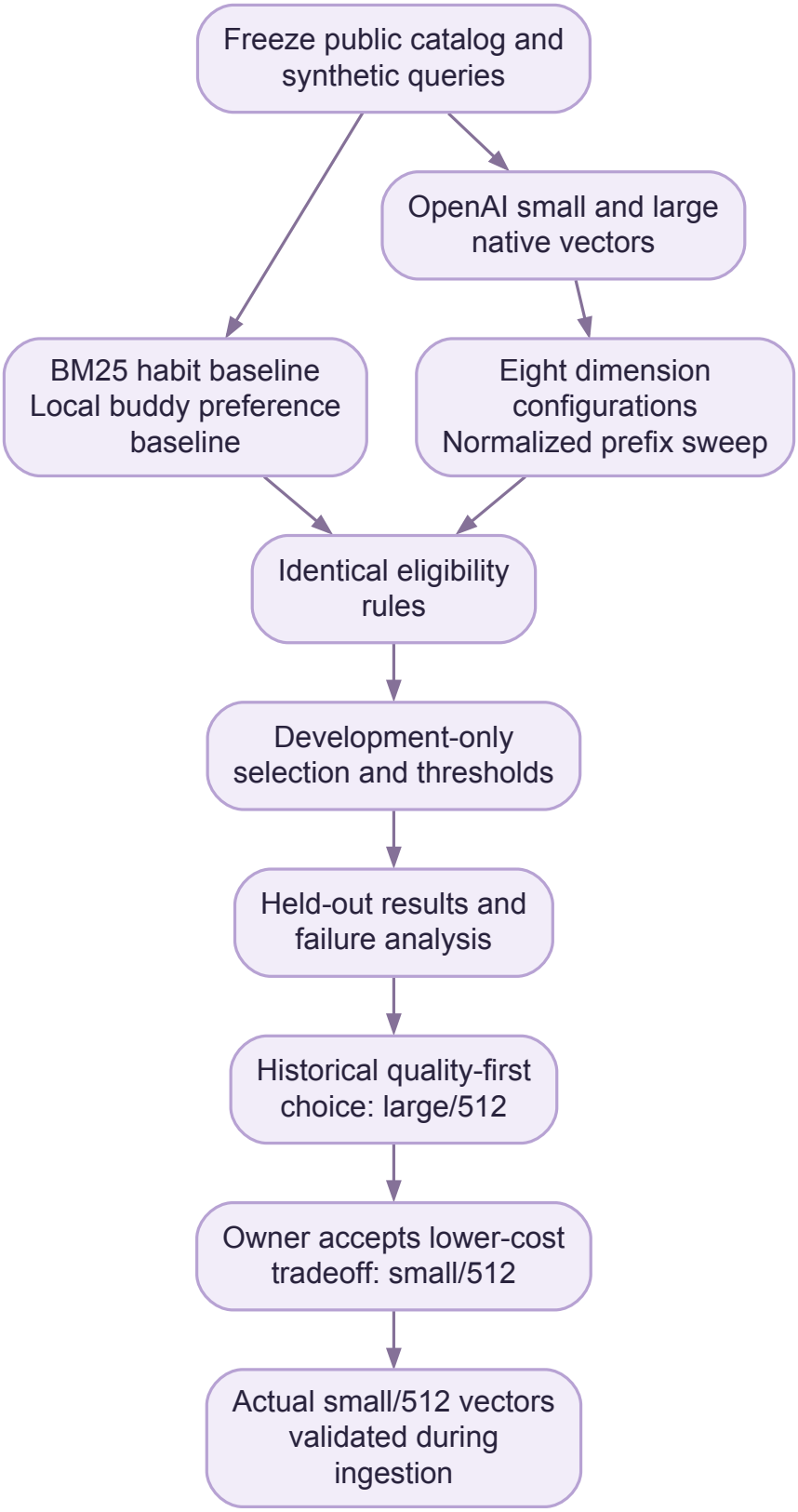
The implemented choice is **OpenAI** `text-embedding-3-small`**, 512 dimensions, cosine similarity**. It retrieves canonical goal candidates from the habit catalog. It does not write advice, generate habit instructions or select buddies. Those responsibilities belong to the AI workflows and local buddy selection.

## Where the research lives

The broader product and behavioral research is in `hq-research`. The executable embedding study, frozen cases, measured results and final model decision are in `hq-infra-setup/embedding-evaluation`. Keeping those locations distinct matters: the child-development literature informed the catalog; the retrieval experiment compared ways of finding its cards. Neither experiment trained a custom model.

## The question we tested

Could a compact embedding retrieve useful age-appropriate cards from ordinary parent wording, at a reasonable cost, compared with lexical search? A separate exploratory question asked whether embeddings improved browsing a catalog of only 24 buddies enough to justify a remote dependency.




The study used exact local cosine search, not eight separate Pinecone indexes. This isolated the embedding comparison from cloud search behavior.

## Dataset and evaluation method

| Item | Recorded method |
|---|---|
| Corpus | 405 public documents: 381 age-specific habit cards and 24 buddies; candidate sets remained separate |
| Cases | 80 synthetic examples: 64 habit requests, 9 semantic buddy-browse requests, 6 buddy controls and 1 buddy choice-set case |
| Split | 40 development and 40 held-out cases; entire scenario families stayed in one split |
| Ages | Four editorial bands, represented by ages 4, 7, 9 and 12; boundary ages were not separately evaluated |
| Labels | Source-linked editorial relevance grades written before viewing rankings; no real-child data or paid model graders |
| Baselines | BM25 over the same habit corpus; deterministic explicit-preference buddy selector |
| Quality | nDCG@5 for useful-card ordering, Recall@5 for coverage of labeled useful cards, MRR for the first useful result |
| Other checks | Unsupported-query behavior, eligibility violations, age/task slices, vector storage, local latency and provider usage |
| Selection discipline | Model/threshold decisions used development cases; family-level bootstrap comparisons are retained in the result JSON |

Age, context, availability and child-card versus parent-guidance eligibility were held constant across approaches. Relevance judgments never entered candidate filters or query embeddings. The held-out habit set had **28 positive cases and 4 unsupported cases**. Its gold labels are incomplete editorial judgments, not an exhaustive assessment of every possible useful card.

See case methodology, frozen cases and machine-readable comparisons.

## Results and the decision

These are the recorded November 1 results **after the experiment's development-calibrated abstention step**. They are not live production metrics.

| Configuration | Held-out nDCG@5 | Recall@5 | MRR |
|---|---|---|---|
| BM25 baseline | 0.738 | 0.579 | 0.929 |
| Small / 256 | 0.733 | 0.563 | 0.964 |
| **Small / 512 - implemented** | **0.727** | **0.571** | **0.964** |
| Small / 768 | 0.709 | 0.551 | 0.929 |
| Small / 1536 | 0.669 | 0.520 | 0.857 |
| Large / 256 | 0.760 | 0.587 | 0.955 |
| **Large / 512 - original benchmark selection** | **0.795** | **0.656** | **0.964** |
| Large / 1024 | 0.814 | 0.673 | 0.946 |
| Large / 3072 | 0.834 | 0.685 | 0.964 |

The frozen quality-first selection rule chose **large/512** on development data. The owner subsequently chose **small/512** for implementation after reviewing the quality, cost and uncertainty. The decision did not relabel the benchmark winner or change its measurements.

The tradeoff was concrete:

- Small/512 and large/512 had the same first-useful-result MRR of 0.964.
- Large/512 had better top-five coverage and ordering. Small/512 also did not beat BM25 on those two aggregate metrics.
- At the rates recorded for the study, small cost $0.02 per million input tokens and large $0.13: about 85% lower embedding-token cost for small. These are dated study inputs, not a fresh pricing claim. Dimensions do not change that token rate.
- Both 512-dimensional choices have the same raw vector footprint. The deployed 381 habit vectors occupy 780,288 bytes as float32, excluding metadata and index overhead. The study's 405-document figure was approximately 0.829 MB.
- The small synthetic set did not establish that paying for large would produce a reliable improvement for real families.

The complete comparison report, recommendation and decision record preserve this distinction.

## What latency and quality evidence do not establish

The dimension sweep shortened native API vectors to a prefix and normalized them. Within that experiment, only the original **large/512** finalist received dedicated actual-`dimensions=512` confirmation and repeated single-query API latency probes. Its reported **261 ms median API latency cannot be assigned to small/512**. Local scoring was approximately 16 ms for either 512-dimensional configuration and excluded the embedding request.

The subsequent ingestion separately validated actual small/512 outputs for all 381 habit texts and four synthetic queries. That proved the implemented vector shape and operational path; it did not rerun the held-out comparison or establish matched small-versus-large API latency.

Every model and BM25 returned cards for one of the four unsupported held-out requests. A similarity threshold is not a safety or suitability decision. The main app therefore retains local urgent routing and its bounded Coach decision and generated-prose review, while retrieval itself performs no model safety call. The one valid query rejected by small/512's study threshold also shows why ranking quality and abstention quality must be assessed separately. The runtime does not simply copy this benchmark threshold into production.

Historical buddy controls included a no-buddy option. That is an experiment record, not the present product contract: HabitQuest now always supplies a default buddy and keeps age-based discovery and explicit choice local.

## Current runtime contract

| Setting or boundary | Implemented value |
|---|---|
| Embedding model | `OPENAI_EMBEDDING_MODEL=text-embedding-3-small` |
| Dimensions | `OPENAI_EMBEDDING_DIMENSIONS=512` |
| Credential/base URL family | `PLANNING_OPENAI_API_KEY` and `PLANNING_OPENAI_BASE_URL`; published deployment config uses `https://api.openai.com/v1` |
| Pinecone target | Index `hq-habits-v1`, namespace `habits-0-1-0`, catalog `0.1.0` |
| Dense search | Cosine; Documents API `2026-07`; field `embedding` |
| Normal retrieval | One name-masked query embedding and one filtered search, followed by local canonical checks |
| Result bounds | Ask Pinecone for at most 12 hits; keep at most 6 distinct canonical concepts for the Coach |
| Failure behavior | Report retrieval failure; do not silently substitute lexical results for a failed live search |

`Settings.validate_planning_features` rejects a model, dimension, index or release mismatch. Equal vector length does not make embeddings from different models compatible. The older hosted-Llama/1024 suggestion is superseded.

The main app has an explicitly selected BM25 fixture mode for offline checks. Although the research recommendation retained lexical search as a possible fallback, production does **not** automatically fall back to it. The current contract is in configuration, embedding client construction, retrieval and the public deployment configuration.

## Costs, reproducibility and future changes

The historical comparison used 33 successful embedding requests, 155,199 input tokens and **$0.01449602** at its recorded standard rates. The later ingestion used 46,066 tokens for 381 cards and 130 for four queries, totaling **$0.00092392**. These are different runs with different scopes; neither total includes Pinecone charges. Per-request embedding and search usage now enter the application's Tokenomics accounting.

Reproduction starts from the frozen corpus/case hashes, preserved pre-publication inputs and the runner guide. The saved experiment passed 23 offline runner tests and 4,230 fixture checks. The ingestion results record the later actual small/512 validation. No evaluation was rerun to write this document.

A future model change needs a new explicit model/dimension decision, compatible catalog and query vectors, a versioned index/release, and independent evaluation. Preserve prior failures and ledgers. More diverse, consented and deidentified parent queries would be needed for stronger product claims; retrieval scores remain distinct from child outcomes.

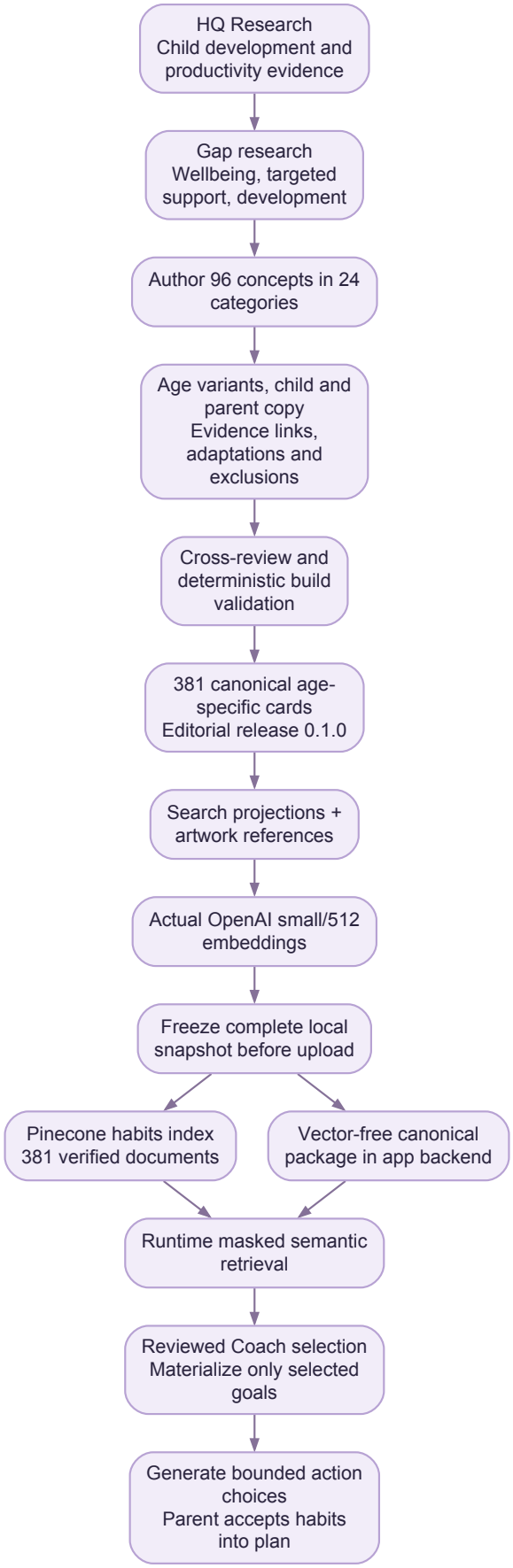
Next: research, catalog construction and Pinecone publishing.

# 10 Habit research, canonical catalog and Pinecone

Reviewed November 6, 2026. The system separates **research**, **editorial catalog creation**, **search publication**, and **family-specific generation**. Research and catalog publication happen outside a chat request. A runtime search finds existing canonical cards; it does not research new papers or rebuild Pinecone.

The central mapping is **catalog habit card -> HabitQuest goal**, then **generated action -> HabitQuest habit beneath that goal**. This preserves the researched goal as a stable source while allowing a small, usable action for the child.

# From evidence to usable goals



# 1. Research foundation and selection process

hq-research contains two connected reviews: child development/behavior and habits/productivity/grit. The work was a structured, purposive product-research review, not an exhaustive systematic review, new meta-analysis or validation study of HabitQuest.

The review prioritized relevant systematic reviews, controlled studies, longitudinal work, mechanism research and official guidance. Topic/author, PubMed/PMC, ERIC-oriented, DOI/publisher and institutional searches provided leads; claims were checked against available original or official sources. Direct OpenAlex/Semantic Scholar API attempts and one Europe PMC route were unavailable and remain recorded as unsuccessful access attempts. Research Scout documents informed the discovery approach; its integrations were not a completed automated research service for this project.

The retained methodology, source hub, source index and search log distinguish full text, manuscript, abstract and excerpt access. They record 58 paper/review records, two guidelines and one published correction, plus six books kept separately. The 78 logged search/access observations are not a screened-study denominator. An independent agent audit checked 15 selected source claims; it was not human clinical peer review or verification of every source.

Three subsequent authoring tracks filled catalog-specific gaps. The assembled catalog library has the following provenance:

| Contribution | Records | Purpose |
|---|---|---|
| Earlier HQ Research library | 61 | Child behavior/development, motivation, habit/grit and critical evidence |
| Wellbeing gap review, W01-W15 | 15 | Care, feeding, sleep, movement, media and household participation |
| Targeted-support review, T01-T05 | 5 | Co-regulation, access needs, worry support and accessible directions |
| Development review, D01-D07 | 7 | Reading, play, learning, communication and safeguarding education |
| Planning synthesis, R01 | 1 | Bounded rationale for linking a cue to an action |
| **Evidence/guidance registry** | **89** | Source records, not 89 independent trials |
| Framework books | 6, separate | Design questions and analogies, not proof of a child intervention |

Reviews can reuse primary studies and each other's populations. Counts must not be added into a new participant total or used as an efficacy score. The evidence supports particular practices or mechanisms in particular settings; it does not establish the exact card, generated action, reward system or app's effectiveness. See catalog evidence methodology.

# 2. Editorial design of the catalog

Each of the three tracks contains eight categories and 32 concepts. Age-specific expansion produces 381 records:

| Age band | Records | Editorial emphasis |
|---|---|---|
| 3-5 | 93 | Adult participation, modeling, familiar objects and shared play |
| 6-8 | 96 | Concrete actions and explicit scaffolding |
| 9-10 | 96 | Shared planning and useful feedback |
| 11-13 | 96 | Negotiated autonomy, privacy and continued support |

Three concepts deliberately omit a preschool variant: comparing a work example, estimating/adjusting a plan, and suggesting a support adjustment. Age bands are editorial groupings; they are not readiness tests or trial-derived deadlines.

The catalog distinguishes 239 general-wellbeing records, 138 targeted-support records and four parent-coaching records. The app's child-goal search permits the first two intent classes; parent-coaching records are not silently turned into tasks for a child.

Authors asked what situation the action addresses, what the adult must provide, what the child can observably choose or attempt, which evidence applies, when to avoid or adapt it, and whether it duplicates another demand. Cross-reviews corrected overlapping concepts, unclear adult/child roles, opt-out wording, consent/access language and overstated research transfer. These decisions are preserved in the resolution record.

The complete card carries more than a title and image:

| Card section | Design purpose |
|---|---|
| Identity and age | Stable source ID, concept/category, age range and readiness note |
| `kid` | Short title, instructions, help phrase and accessible delivery options |
| `parent` | Selection, setup, coaching, adaptations, review and things to avoid |
| `practice` | Observable action, support/adult role, easier version and time estimate |
| `safety` | Specific exclusions, unsupported uses and access boundaries |
| `research` | Source IDs/links, insight, evidence relationship and explicit limits |
| `stacking` | Context, cue, materials, effort and possible observed output events |
| `retrieval` | Prepared search text, phrases, language and availability |
| `presentation` | Editorial display information; artwork is a separate overlay |

Evidence entries retain `supports`, `does_not_establish` and `age_transfer`. Relationships such as `intervention_component`, `mechanism_informed` and `practice_guidance` explain why a source is relevant; they are not certainty scores. A boundary citation may explain why clinical treatment findings must **not** be applied to an ordinary action. Source detail views must retain that qualification, rather than treating every citation as positive proof.

## 3. Canonical build and preservation

The editable sources are in `hq-habits-catalog/authoring`. The local `rebuild.py` runs catalog, stack, schema and validation builders in dependency order without network or database calls. It produces canonical JSON, category views, readable cards, schemas, search projections and validation records together.

`data/habits.json` is the complete canonical delivery. Category files are alternate views of the same IDs, not additional databases. Do not hand-edit one generated projection and leave the others stale. The validation report checks schema/reference consistency, age/context bounds, source links, both hash types, projection parity and important conditional behaviors. Its 48 lexical smoke queries are not a semantic retrieval benchmark or child usability study.

The source catalog also contains 14 stack templates, 56 age-specific resolved recipes and 95 editorial relations. These are preserved supporting data, not extra Pinecone documents or automatic app assignments. Similarity does not establish sequence, readiness or whether a proposed event actually happened. The app's Quest Planner must not be confused with these catalog stack recipes.

The catalog's reward policies were not imported into HabitQuest. The adapter removes reward/point fields from evidence exposed for generation, and the app retains its existing points, Sparks and rewards. Catalog observations about reward gates therefore must not be described as fully implemented app behavior.

## 4. What was published to Pinecone

The selected deployment is a managed Pinecone index on AWS `us-east-1`, using externally generated OpenAI vectors. Pinecone does not run a hosted embedding model for this index.

| Contract | Published value |
|---|---|
| Index / namespace | `hq-habits-v1` / `habits-0-1-0` |
| Catalog release | `0.1.0` |
| Documents | 381, one per age-specific source card |
| Embedding | `text-embedding-3-small`, actual API `dimensions=512` |
| Search interface | Documents API `2026-07`, dense field `embedding`, cosine |
| Identity | Source ID plus concept/category/release and canonical/search-text hashes |
| Metadata | Age/context/availability/language/intent, source references and artwork references |

The setup configuration and ingestion approach define the payload. Prepared search cards are already small semantic units; there is no arbitrary paper/PDF chunking service on this path.

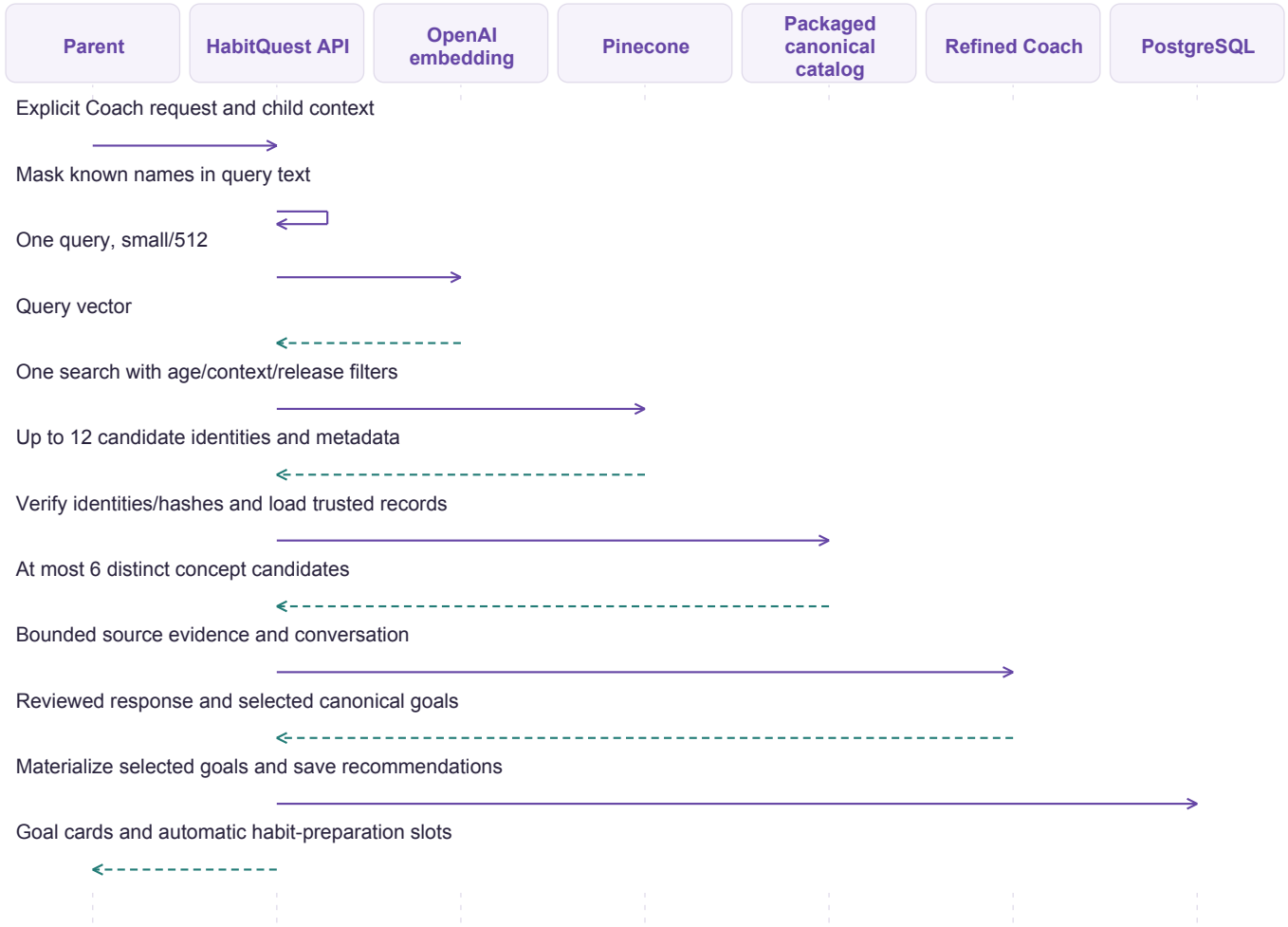
Publication proceeded in this order:

1. Validate public canonical records and projections; pin input/configuration hashes.
2. Generate actual small/512 vectors for 381 search texts in six API batches.
3. Preserve and read back a complete frozen local snapshot **before** creating the index/namespace or uploading the payload.
4. Upload six batches of the exact archived documents.
5. Fetch and compare all 381 stored IDs, text, metadata, S3 references and vectors; verify namespace count and run bounded synthetic retrieval checks.

The snapshot ID is `22cec20089fee49e76ee76861b01a199e05f765ec19a2f1911eddef1088828df`. `datasets/current.json` points to its frozen directory. The archive includes complete canonical records, embedding text and exact vectors, upload JSON/JSONL, plus supporting buddies, stacks, categories, age definitions and artwork overlays. Those supporting collections were not embedded or uploaded as habit-search records.

The November 1 execution results record successful full-fetch verification and four operational searches spanning ages 4, 7, 10 and 12. Two negative filters returned no rows. Strict search-score equality did **not** pass: maximum observed difference from local exact cosine was 0.0003983 against the original 0.0002 diagnostic tolerance. Stored-vector integrity and returned candidate sets passed; the numeric variation remains explicit, without claiming its precise cause was established.

## 5. How the main application uses it




The backend bundles a **vector-free** canonical snapshot in `planning_catalog.json`. `package_planning_catalog.py` verifies the frozen source manifest and record/vector hashes before creating this portable package. A deployed task needs neither a sibling checkout nor cloud access to read complete source guidance.

`CatalogRepository` validates the package checksum, snapshot, model/release contract, original record hashes and metadata. It creates an application goal ID as `hqg-` plus the first 24 hexadecimal characters of SHA-256 over the source ID. The source reference keeps the original source/concept/category, release, full-record hash and snapshot.

`CatalogRetriever` embeds the already name-masked text, filters age, setting, availability, language, release and allowed child intent, and excludes previously requested concepts when appropriate. It verifies returned identities and metadata against local source facts, then keeps at most six distinct concepts. It does not run a safety model, web research, model-based reranker or database import during retrieval. The refined Coach's output decision/prose review is a separate later step.

Only selected goals from a reviewed Coach recommendation are materialized in PostgreSQL. Browsing source details and similarity search do not preload the 381 goals. The restricted materialization function verifies child ownership and the pinned canonical payload; source collisions/changed records fail instead of silently overwriting data. Habit generation requires that reviewed goal mapping. An explicit owner import tool remains available for named goals, but is not the normal startup/runtime flow.

Generated actions then become draft habits under that exact goal. The app normally prepares three choices per newly returned goal, at most four goals and two batches in flight. Parent acceptance is still required before assignment. See Coach and habit generation and database design.

## Storage responsibilities and design boundaries

| Store | Owns | Does not own |
|---|---|---|
| Research/authoring folders | Evidence, editorial decisions, source data and build history | Live family plans |
| Frozen ingestion archive | Reproducible source release, vectors and exact upload payload | Mutable request history |
| Pinecone | Semantic retrieval over public habit source documents | Family/child records, habit assignments or source authority |
| Packaged canonical JSON | Complete verified instructions, research and source provenance | Mutable family state or remote query vectors |
| PostgreSQL | Selected goal mappings, recommendations, generated habits, plans and usage | A bulk copy of the vector index |
| Artwork manifests / S3 / packaged frontend assets | Independently versioned visual resources | Evidence of efficacy or embedding values |

The separate artwork overlay can supply real images even where frozen editorial records still say `asset_status: not_generated`. The completed art manifest, not that historical placeholder, determines asset availability. See buddies and assets.

## Maintenance and remaining evidence limits

Changing search text, full source guidance, artwork or embedding space are different operations. `embedding_text_sha256` identifies search-text changes; `canonical_record_sha256` detects guidance/evidence changes even when search text is unchanged; art versions/hashes track media separately. A future release must create new preserved inputs and compatible query/index contracts rather than overwrite frozen evidence or bypass application hash checks.

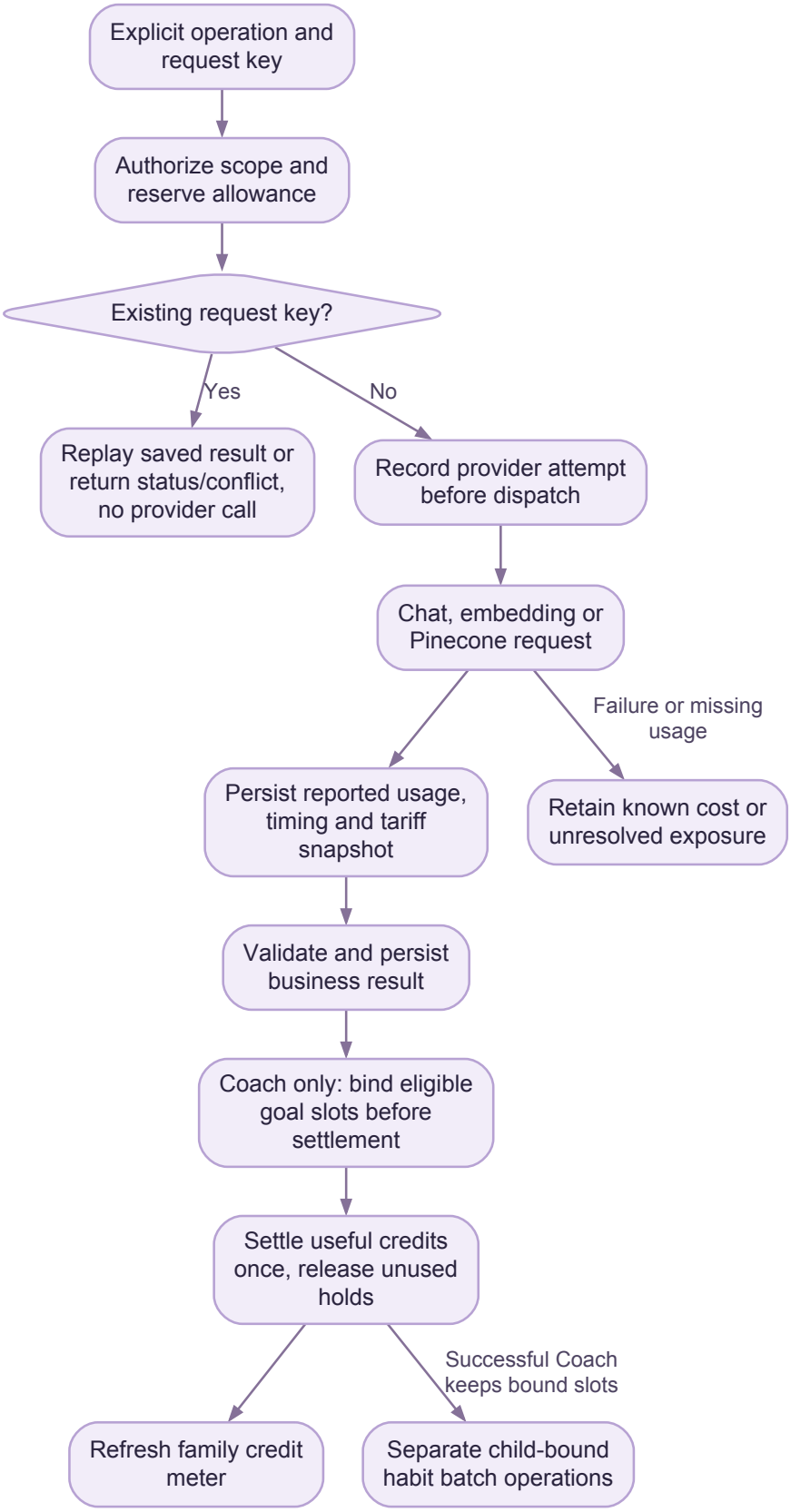
The older catalog and infrastructure READMEs contain milestone statements such as “not ingested” or “service unimplemented.” Those describe their original authoring stage. Later ingestion receipts and the present main-app implementation supersede them for deployment status. The standalone planning-agent prototype is the source of adapted retrieval/catalog/generation capabilities; it is not a second runtime service called by the deployed HabitQuest Coach.

No new papers, embeddings, database writes or Pinecone calls were made for this documentation review. Research remains selective and access-constrained; the catalog and retrieval evaluations do not establish clinical approval, family usability or product efficacy. The embedding selection document separates measured retrieval results from the accepted implementation tradeoff, and AWS deployment describes current hosting.

# 11 Tokenomics, latency and observability

Tokenomics separates what a family experiences from what the operator pays. Family credits describe useful completed activities. The provider ledger records actual HTTP attempts, including embedding/search usage, retries and failures. Progress stages and timers describe where elapsed time went. These are related measurements, not interchangeable counters.

## Accounting model




`generation_metering.py` wraps existing operations. `tokenomics.py` owns admission, holds, replay and settlement. Business text stays in domain tables; accounting stores IDs, statuses and numeric observations rather than copying family messages or model bodies.

An operation represents a useful user activity. An invocation is a logical provider call. An attempt is an actual HTTP dispatch; SDK retries are separate attempts. The database records exposure before dispatch, outside the transaction for the business result, so a later rollback does not erase provider usage.

## Family allowance and operator policy

The seeded policy below is the implemented starting configuration, not a claim that an operator cannot edit it later.

| Activity or setting | Starting value | Behavior |
|---|---|---|
| Daily allowance | 100 credits per family-local day | Shared across children; reset follows family timezone |
| Useful Coach response | 2 credits | Includes retrieval; not a separate fee per returned goal |
| Useful habit batch | 1 credit for up to three choices | Automatic and explicit batches have separate child operations |
| Quest Planner, Weekly Review, onboarding mapping | 2 credits each | Useful result settlement, not every HTTP attempt |
| Buddy / child-note screening | 0 family credits | Provider cost is still recorded |
| Enforce budgets | Off | Demo mode tracks usage and overage without blocking on allowance/cap |
| Show generation costs | Off | Family dollar fields and supported detailed UI are opt-in |
| Reference USD per credit | $0.0015 | Input to recommendations for activity weights, not a real-time currency balance |

Four fresh goals normally mean 2 Coach credits plus four one-credit batches, or six credits. Retries may increase operator cost without multiplying useful family debits. Safe local replies and failed results do not receive a useful-result debit; incurred provider cost can still exist. Idempotent replay does not add one.

Policy edits are revision-checked and audited. Price changes create new tariff versions; historical attempts keep their price snapshots. Enabling enforcement uses existing exposure, including imported legacy reservations. It does not reset the old cap or create a clean ledger. Unknown/unbounded exposure can block new dispatches in enforcement mode. The old caregiver cap endpoints are retired.

## Embedding and similarity-search costs

For a single attempt, known cost combines applicable components:

```
chat = uncached_input_tokens x input_rate
     + cached_input_tokens x cached_rate
     + cache_write_tokens x write_rate, where separately reported/applicable
     + output_tokens x output_rate
embedding = embedding_input_tokens x embedding_rate
search = returned_read_units x read_unit_rate
       + attributable_egress_bytes x egress_rate
operation estimate = sum of its own attempt estimates
recommended activity credits = max(1, ceiling(complete reference cost / USD per credit))
```

Rates use their stated denominators, usually one million tokens/read units or a specified byte unit. Cached/reasoning counters must not be added again to provider totals. Automatic habit operations are separate rows, so a Coach interaction total must not add both those rows and an already aggregated interaction total. If a complete reference cost is unavailable, the recommendation is unavailable; it is not fabricated from only the known components. Policy weights for zero-credit activities remain an explicit operator decision.

The November 5 analysis recorded list-rate inputs of $0.10 input / $0.50 output per million Luna tokens, $0.30 / $1.20 for the configured DeepSeek model, and $0.02 per million embedding input tokens. These are **dated analysis inputs**, not a newly verified price quote or an invoice. See the original rate sources and assumptions.

Pinecone is not free merely because its marginal cost is small. Its read units and applicable egress are included. The seeded tariff uses a planning assumption; the real account plan, minimums, storage charges and free allowances were not verified by this implementation. Missing usage is unknown, not zero. See retrieval arithmetic.

## Timing boundaries

| Measurement | Boundary | Interpretation |
|---|---|---|
| Provider attempt | Actual HTTP request to response/error | Includes network/provider delay for that attempt |
| Generation stage | Server-reported named stage | Separates retrieval, writing, review and validation work |
| Server operation | Endpoint work and result handling | Does not include later independent habit batches |
| Browser response | Request initiation to usable final response | Includes transport and client processing |
| All choices ready | Coach interaction through queued habit batches | Longer than text latency; batches overlap at concurrency two |

`generation_progress.py` emits fixed stage names and elapsed time over SSE, followed by the same final authorized result. It does not expose unreviewed token-by-token model drafts. Disconnecting a browser does not undo an already dispatched provider call. `generationPreferences.ts` keeps account-scoped local diagnostic history and the optional seconds display; server operation records retain timings and browser observations separately.

Historical useful samples include an 11.727-second Coach HTTP request and two three-choice batches around 10.35-10.71 seconds on November 5. Earlier narrower November 4 samples were faster. Different output sizes and ASGI/HTTP boundaries must not be pooled into a production mean or p95. No representative cloud generation-latency benchmark was established by the November 6 health checks.

## Operator and user surfaces

- `/admin` has a separate operator login and token. It configures policy/prices and shows cohorts, operations, models, usage and cost status; it does not browse family conversation content.
- Generation history includes input/output tokens when reported, with **Measured**, **Partial**, **Not measured** and **Not applicable** distinctions. Embedding input is counted as tokens; Pinecone read units are not tokens.
- Dashboard and Coach retain the compact allowance chip, including demo/enforced mode and overage. Completion/failure events refresh the authoritative balance.
- The current Coach omits inline credit/prepared-habit messages and per-batch quotes. Dashboard report details follow the operator visibility flag. The shared accounting component used by some other views still has its own credit rendering; do not assume every surface has one universal visibility gate.
- The family seconds toggle is independent of operator cost visibility. Turning off a display does not stop metering or change budget enforcement.

The November 6 fix added refined successful outcomes to the settlement allowlist. Previously valid recommendations were marked failed, settled at zero and had automatic habit slots canceled. The corrected path records useful credits and keeps those slots. It did not backfill old zero-credit history. An existing browser tab can retain older UI code until refreshed; that was verified on the deployed app. See the release and follow-up investigation.

## Operational limits and sources

Always distinguish actual measured usage, estimated dollars, conservative reserved exposure and family credits. Readiness checks are not model entitlement or invoice checks. Tracing, when enabled, must respect privacy; accounting itself excludes prompts, response text, vectors and keys. Scaling beyond the present one-worker demo also requires shared handling for in-memory throttles/caches.

- Tokenomics operations
- Attempt instrumentation
- Family projections, operator API
- Admin UI, budget UI
- Metrics migration

# 12 Buddies, catalog artwork and asset delivery

The final presentation design uses the richer catalog's public artwork and metadata while preserving HabitQuest's existing identity, wardrobe and reward rules. No LLM, embedding or Pinecone request is needed to recommend or select a buddy. This document describes the November 6, 2026 implementation.

## Asset families and where they live

| Asset family | Current representation | Runtime delivery |
|---|---|---|
| Original buddies | Fox, Owl, Bear and Bunny SVG artwork | Bundled React/SVG code; local fallback remains available |
| Catalog buddies | 24 named characters in release `0.1.0`; 48 PNGs, one 128px portrait and one 768px full view each | Versioned, content-addressed public HTTPS S3 URLs in the packaged manifest |
| Goal and category artwork | Presentation metadata for 381 goals and 24 categories, with sharp originals and smaller variants | Copied into `frontend/public/catalog-art`; served with the frontend container |
| Generated habit icons | Persisted icon keys mapped to a bounded Fluent-style set | Packaged SVGs under `frontend/public/fluent-emoji` |
| Wardrobe accessories | Eight existing SVG accessory designs | Composed over original or catalog buddy artwork using local code |

The buddy image URLs use the public prefix `https://habbit-quest-assets.s3.us-east-1.amazonaws.com/buddies/` followed by pinned catalog/art versions and a content identifier. The browser does not list a bucket, fetch AWS credentials, upload artwork or request presigned URLs. Anonymous-read verification was recorded during the buddy rollout; this documentation work does not make a new claim about every object or policy in that bucket.

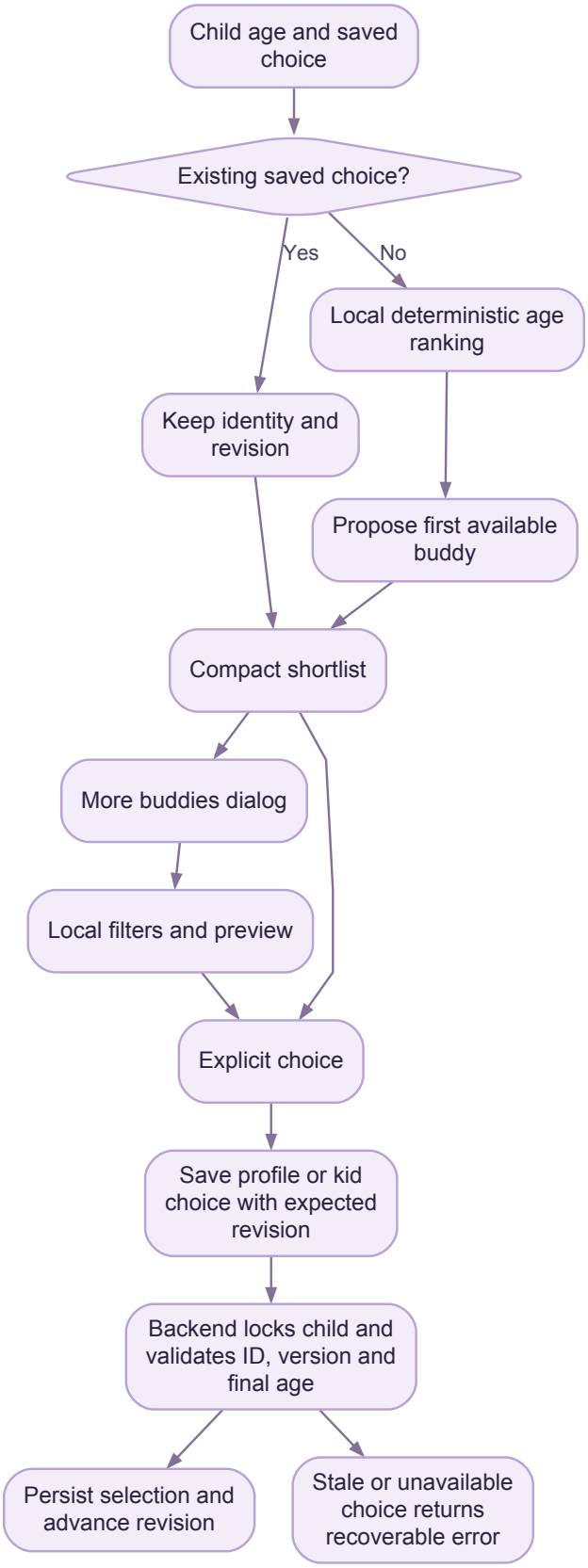
The source catalogs remain in the surrounding workspace. Runtime manifests are deliberate public projections, not a way to embed private research notes or family data in the browser.

Sources: buddy source catalog, frontend buddy manifest, catalog presentation, catalog asset manifest, recorded S3 checks.

## Every child keeps a buddy

There is no “None” buddy option. Existing children preserve their saved original buddy unless someone explicitly chooses another. New-child setup proposes a deterministic age-based catalog default; age edits can update that provisional default until the caregiver deliberately chooses a character. The submitted profile confirms the selection.

All 24 current catalog characters support ages 3-13. Their primary age bands are soft suggestions, not an exclusion rule or a developmental diagnosis. The chooser explains that children may select any supported character.




The ranking implementation supports explicit interest/tone filters: three points per matched interest, two for tone and one for the primary age band. Stable ID ordering and group diversity make the shortlist repeatable; explicit exclusions remain filters. In normal age-only setup, the score is a transparent local presentation rule. The app does not infer a child's interests, personality or behavioral profile from the buddy.

Sources: ranking, child form, chooser, backend selection checks.

## Efficient selection and metadata display

The new-child form shows three choices. Child Details can show six; the kid chooser keeps the saved character plus two alternatives. The full dialog provides the remaining catalog and original buddies, local theme/interest filters, a larger preview, Cancel and an explicit Choose action. Browsing or previewing does not persist a selection.

Name, concise characteristics and suggested ages are shown while choosing. Detailed descriptive panels are not repeated throughout the dashboard or daily quest screens. Those surfaces retain the saved avatar/name and neutral event copy. The metadata is not passed to Coach, habit selection, scoring or rewards.

Portraits serve small controls; the selected larger preview requests the full image. Grid images can lazy-load, and images decode asynchronously. The selected image element changes with its URL so an old character does not remain painted under a newly selected name. Accessible radio choices, keyboard movement and the shared dialog keep selection usable without a pointer.

## Identity and backward compatibility

The existing `children.buddy` value remains one of `fox`, `owl`, `bear` or `bunny`. Catalog support adds fields instead of replacing that contract:

| Field | Purpose |
|---|---|
| `buddy_selection_mode` | `legacy` or `catalog` |
| `catalog_buddy_id` | Public canonical character ID in catalog mode |
| `buddy_catalog_version` | Pinned metadata release |
| `buddy_selection_revision` | Advances on deliberate choice; guards conflicting saves |
| Existing `buddy` | Original visual/theme fallback |

Parent and kid writes validate the same saved identity and age. Kid writes use a narrow database function rather than broad permission to edit child rows. The frontend merges incoming buddy state only when its revision is current or newer. Old unrevisioned profile writes cannot undo a newer explicit choice.

Missing metadata, an image error, disabled catalog selection or an empty filter does not clear the child's identity. Rendering falls back to the original local SVG while retaining the catalog ID/version/revision. Turning off the chooser is a presentation rollback, not a data migration.

Sources: identity adapter, buddy renderer, additive migration.

## Outfits use real overlays

The final design preserves party hat, crown, shades, bow, wizard hat, flower, headphones and halo. It does **not** replace catalog outfits with item badges. The existing accessory SVGs are drawn over the generated bitmap using independent portrait/full-body landmarks for each character: eyes, head top, width and, where needed, ears and fitting offsets. The item determines the transform; special fits handle shapes such as robot ear hardware.

The catalog bitmap itself is unchanged. Earned inventory and the equipped item remain in `child_extras`; changing a buddy does not erase the wardrobe. “No accessory” removes only the worn accessory, never the buddy. Local SVG fallback also preserves the equipped item. The rollout reviewed 24 characters x eight accessories x two views = 384 combinations, with the original SVG renderings preserved.

Sources: overlay composition, landmarks, outfit review.

## Goal and habit artwork

Goal cards retain category, age, setting, support and research context, with a compact preview and a larger details dialog. `CatalogVisual` looks up public presentation by canonical source or goal ID. Responsive image choices prefer the 512px artwork above thumbnail size, avoiding the earlier hazy enlarged thumbnail problem. A failed image has a local emoji fallback.

Accepted habits retain their exact linked goal. Child detail views show a concise goal reference image alongside the child-facing action steps; they do not expose grown-up notes. Generated habit icons use the saved `emoji_key` and a local mapping, so rendering does not make an image-generation call or require a generated bitmap for every action.

Public metadata and artwork are presentation only. Displaying all available goal images does not bulk-import all source goals or add anything to a child's plan. See database boundaries and Coach design.

Sources: responsive goal artwork, habit icons, child-facing detail view, catalog asset provenance and sizing.

## Packaging and validation

Frontend buddy packaging verifies published asset identity, dimensions, byte counts and hashes before projecting metadata. Backend packaging produces the small pinned reference used for age/version validation. The backend migration contains the same public identity/age records; applied migrations remain immutable.

The recorded implementation checks cover defaults, deliberate cross-band choices, parent/kid saves, revision races, refresh, all accessories, inaccessible images, desktop/mobile dialogs and reduced motion. Tests prove that selection does not invoke providers. Catalog buddy event text is deterministic; the separate optional AI-line path for original buddies remains governed by its existing setting.

Sources: frontend buddy packager, backend buddy packager, backend tests, frontend selection tests, browser scenarios, implementation and verification record.

# 13 Final AWS deployment design

This describes the completed development/demo deployment recorded November 6,

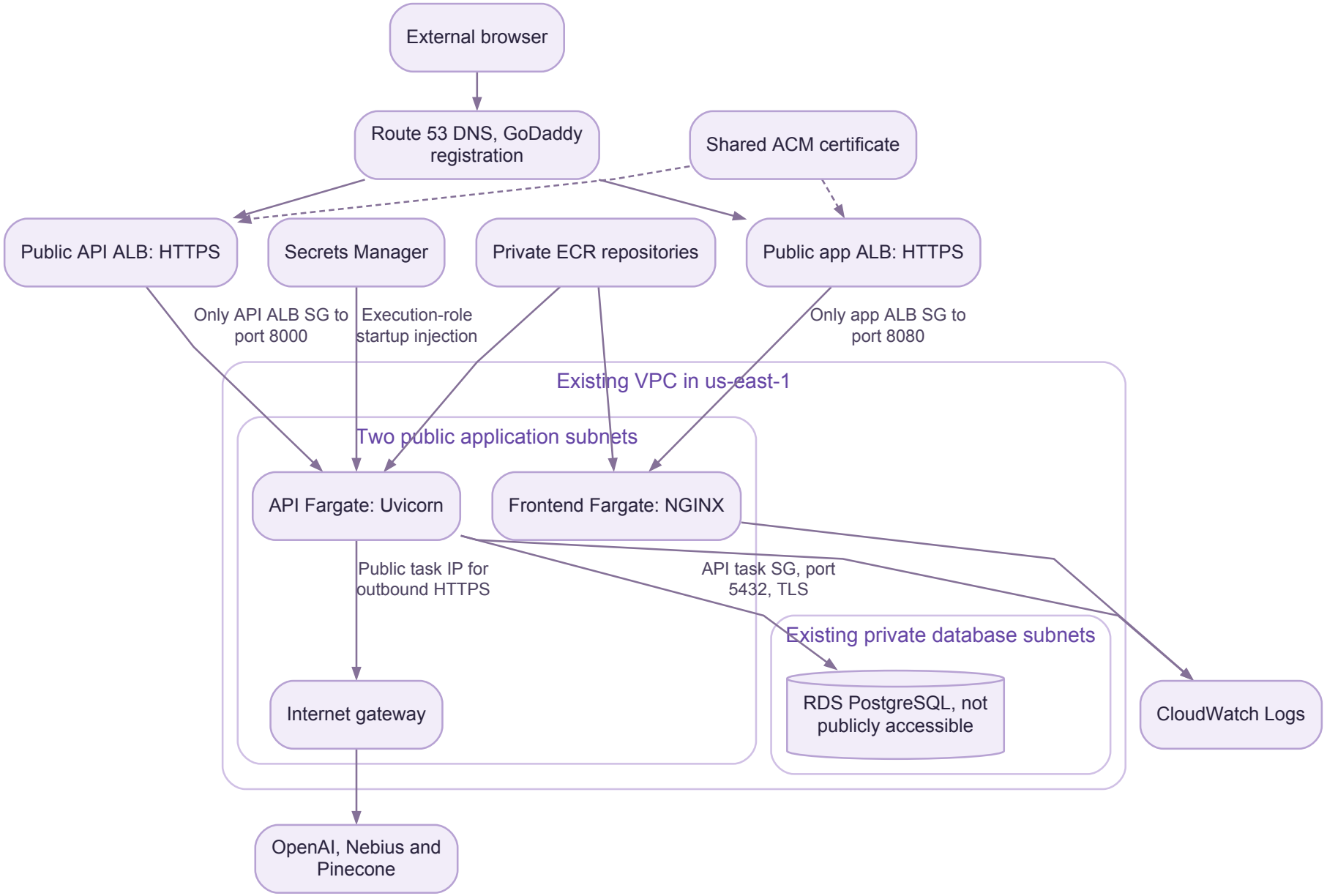
2026. The deployment workspace remains `hq-deployment`, which owns the scripts, CloudFormation, private configuration locations and immutable execution evidence. This document consolidates that design; it does not execute a deployment.

## Public endpoints and traffic

| Endpoint | Purpose |
|---|---|
| `https://habitquest.quest` | React SPA served by the frontend Fargate service |
| `https://habitquest.quest/admin` | Separate operator login in the same frontend |
| `https://api.habitquest.quest` | Public FastAPI endpoint used directly by browsers |
| `https://api.habitquest.quest/health/ready` | Database-backed readiness check |
| `https://www.habitquest.quest` | Redirect to the canonical apex |

The domain is registered with GoDaddy and delegated to Route 53. One ACM certificate covers the configured app/API/redirect names and is shared by two separate ALBs. HTTP defaults redirect to canonical HTTPS hosts. HTTPS exact-host rules forward requests; unknown HTTPS hosts are rejected. The API allows the exact production frontend origin through CORS and still requires the appropriate application token on protected routes.

# AWS deployment and network boundaries



The Fargate tasks are **in public subnets with public IPs**, not private-subnet tasks behind NAT. Inbound service ports accept traffic only from the respective ALB security group. Public IPs provide inexpensive outbound access; no NAT gateway was added. RDS remains private with SQL ingress scoped to approved clients. This distinction is important when explaining the demo's cost/security tradeoff.

## Runtime footprint

| Component | Recorded configuration | Tradeoff |
|---|---|---|
| ECS cluster | `habitquest-dev`, Fargate on demand | Existing recovered platform stack; no Spot dependency |
| API service | One Linux x86-64 task; 0.5 vCPU, 1 GiB, port 8000 | Always running, limited demo capacity |
| Frontend service | One Linux x86-64 task; 0.25 vCPU, 0.5 GiB, port 8080 | Static SPA delivery by NGINX |
| Database | RDS PostgreSQL 18.6, Single-AZ `db.t4g.micro`, 20 GiB encrypted gp3 | Always-on 1 GiB development database, not Aurora Serverless |
| Logs | Retained groups, seven-day retention | Short demo troubleshooting window |
| Application load balancers | Two public ALBs | Owner selected separation over a cheaper single-ALB alternative |

Each service maintains one healthy task while rolling to its replacement with 100% minimum / 200% maximum healthy capacity and an ECS rollback circuit breaker. One steady-state task and Single-AZ RDS are not a high-availability production design. They avoid intentional scale-to-zero/database auto-pause but cannot promise zero latency after failure, restart or provider delay.

Containers run as non-root with read-only root filesystems, dropped capabilities and a writable `/tmp` volume prepared with correct ownership. Task shutdown has a grace period. ECS Exec is not enabled. Neither container runs migrations or catalog seeding on startup.

## Configuration and secret boundary

| Surface | Source | Rule |
|---|---|---|
| Local backend | Ignored `backend/.env` | Settings are read as environment variables |
| AWS backend secrets | ECS execution role reads selected Secrets Manager JSON keys | Inject `JWT_SECRET`, provider keys, admin credentials and `PGPASSWORD` at task start |
| AWS backend public settings | Versioned deployment configuration/task definition | Models, API modes, index contract, origins and feature flags are explicit |
| Frontend | Four public Vite build values | API URL, Google client ID, developer-login flag and buddy flag only |
| OAuth client secret | Private local env and dedicated retained Secrets Manager object | Stored for ownership continuity; current GIS ID-token flow does not consume it |

The seven injected backend secret fields are `JWT_SECRET`, `OPENAI_API_KEY`, `PLANNING_OPENAI_API_KEY`, `PINECONE_API_KEY`, `ADMIN_PASSWORD`, `ADMIN_JWT_SECRET` and `PGPASSWORD`. Runtime code does not call Secrets Manager to fetch them itself. Changing a secret requires replacement tasks to load the new value. The frontend receives no provider, database or admin secret.

Google sign-in obtains an ID token in the browser. The API verifies its audience and Google signature, then issues the app's caregiver token. It does not perform an authorization-code/client-secret exchange. Production origins use the apex; the approved client also permits the documented local port 5180 origins. See OAuth setup and private storage references.

## Database setup and migration boundary

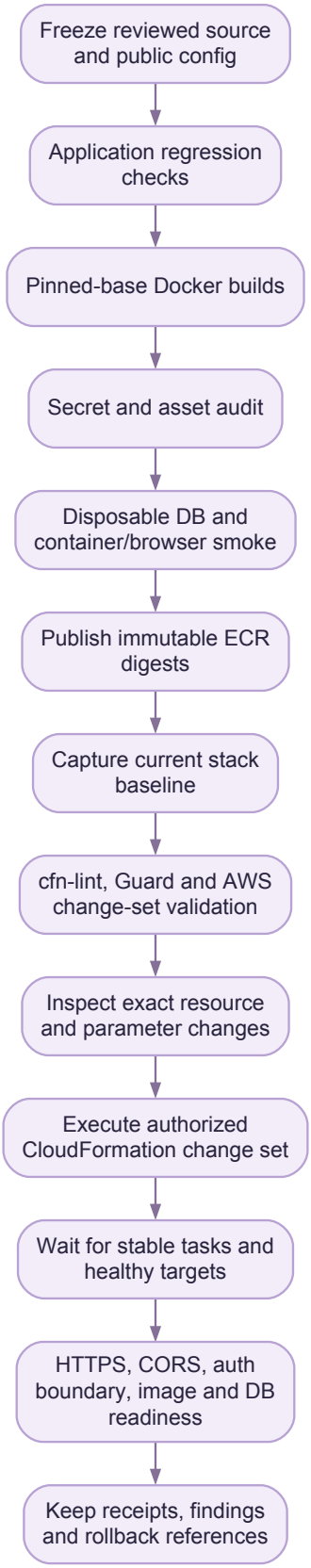
The local safety backup is retained under `hq-db/backups`, with setup documented in the deployment workspace. Import used a CloudShell VPC environment in an existing subnet, a scoped client security group and private encrypted S3 staging. It did not require making RDS public or creating an SSH relay.

The successful restore verified 36 application tables plus the migration ledger, row fingerprints and all 22 migration hashes. The application connects as `hq_runtime`, a non-superuser NOINHERIT login able to assume only the intended application roles. TLS uses `verify-full` and the packaged RDS CA.

The runtime role cannot read the operator migration ledger. Later image verification matched packaged migration hashes to the verified restore manifest; this was not a fresh privileged query of the live ledger. No privilege was widened to perform that check. Direct laptop access still requires a private network access path; the public app/API ALBs do not expose PostgreSQL.

Sources: database import, connection guidance, runtime verification.

## Repeatable release process



| Script/template group | Role |
| --- | --- |
| Build helper, Docker/NGINX inputs | Allowlisted contexts, pinned bases, source fingerprints and public Vite configuration |
| Image audit | Verify packaged assets, credentials boundary and expected data |
| Container verification | Disposable PostgreSQL, restricted runtime, family/kid isolation, synthetic AI and browser checks |
| Publisher | Temporary ECR login, immutable tags/digests, remote filesystem/configuration comparison |
| Release driver | Frozen plan/validation/prepare/inspect/execute/status receipts |
| Current image-only preparation | Require the previous verified baseline and change only image parameters |
| Public verification | Exact task image, health, TLS, CORS, static bundle, auth boundary and Google public-ID consistency |
| CloudFormation directory | Network, security, database, service and domain templates plus Guard rules |

Use the container runbook, ECS runbook, and latest release commands. The scripts deliberately refuse stale baselines and overwritten evidence files. For another release create fresh reviewed inputs/receipts; do not replay an old execution receipt or silently replace frozen evidence. Network/security/resource changes retain the owner's phase-specific approval requirement.

## Last verified application release

The November 6 Coach correction completed on API task revision 4 and frontend revision 5. Both services had one healthy task, no pending tasks and completed rollouts. The two existing service stacks were `UPDATE_COMPLETE` and protected:

- API: `habitquest-dev-ecs-api`.
- Frontend: `habitquest-dev-ecs-frontend-recovery`.
- Platform: `habitquest-dev-ecs-platform-recovery` owns the selected cluster/logs.

The `recovery` names record earlier first-use deployment failures; they do not mean a second active frontend/platform is serving the app. Original failure records were retained. Exact image digests and checks are in publication evidence and public verification.

The frontend scan had no findings; the API retained 9 high / 10 medium / 3 low base-image findings, with no critical findings. The earlier exposure review and remaining remediation are recorded in HTTPS execution. These are dated scan results, not a claim of vulnerability-free software.

Rollback uses newly reviewed change sets restoring prior immutable images and compatible configuration. It does not restore the database or reset Tokenomics. Schema rollback would require a separate data/migration plan.

## Browser update behavior

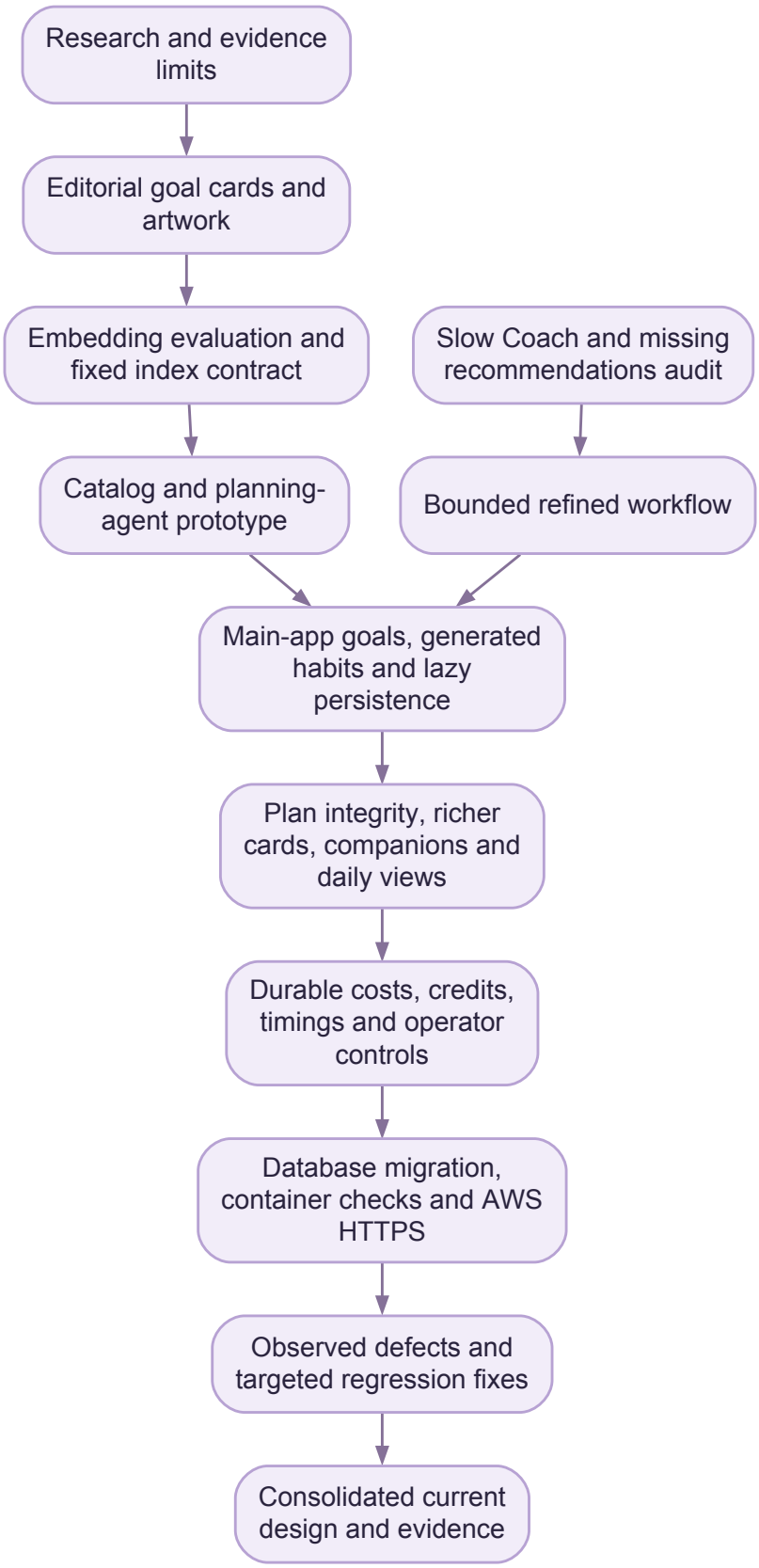
NGINX serves HTML with `no-cache` and fingerprinted JS/CSS with immutable caching. An already-open SPA retains its loaded JavaScript until refreshed; changing routes does not fetch a replacement bundle. The November 6 visibility investigation verified an ordinary reload removed old Coach credit copy while preserving saved goals and habits. Local Vite's automatic code updates are not a production feature.

Public health checks prove route, container and database readiness. They do not prove model key entitlement, generation quality, generation latency or a new Google account sign-in. No paid provider calls were made for the latest release verification, and this documentation task makes no new cloud claims beyond its recorded evidence.

# 14 Design decisions, verification and limits

This record connects the design process to the implemented November 6, 2026 snapshot. It distinguishes a reasoned choice, a completed code change, a measured sample and an operational verification. They provide different kinds of evidence.

## How the design was developed




The process did not treat the earliest proposal as the final specification. Owner decisions narrowed safety scope, rejected importing catalog rewards, required on-demand goal persistence, separated planning and kept demos usable without enforced budgets. The final documents preserve those decisions and their limits.

## Decisions that shape the current system

| Decision | Reason and resulting behavior | Evidence |
|---|---|---|
| Keep frontend/backend together | One main app owns API/UI contracts; the old standalone backend is historical | Repository guide |
| Use canonical cards as goals, actions as habits | Retain source evidence while generating small child-specific actions | Integration plan, current data model |
| Import goals only after reviewed generation selects them | Retrieval/browsing should not bulk-populate the local goal database | Catalog import, research-to-runtime design |
| Replace the multi-call crew with a bounded graph | Reduce unnecessary model work while preserving explicit stages and structured contracts; the earlier crew already had some parallel stages | Refined graph, AI orchestration |
| Review new Coach prose, trust validated canonical context | Keep retrieval fast; avoid repeated model review of researched cards | AI operations |
| Keep habit checks local and acceptance explicit | Bounded actions still require source/schema/age/privacy/novelty checks and parent choice | Habit generation |
| Separate explicit planner and review | History-based planning is a distinct user activity, not work hidden inside every chat | Other capabilities |
| Choose small/512 after quality comparison | Owner accepted lower embedding cost despite weaker top-five metrics than large/512 | Embedding decision |
| Preserve rewards and approved instructions | Catalog reward recommendations must not rewrite the app economy or previously accepted behavior | Database, research policy boundary |
| Keep buddies local with a default | Small catalog, transparent age suggestions, no extra AI cost and no empty identity | Buddy design |
| Meter attempts separately from useful credits | Failed/retried provider work can cost money; replay must not debit the family twice | Tokenomics |
| Use always-on micro RDS and one task per service | Avoid intentional idle pause/cold-start behavior for the limited demo | Database decision, AWS design |
| Retain two ALBs and public-IP tasks | Owner chose separate app/API endpoints; scoped task ingress and direct egress avoid NAT for the demo | Two-ALB proposal, AWS design |
| Publish immutable images through CloudFormation | Verify exact artifacts, inspect change scope and retain rollback evidence | Latest release |

## Recent corrections included in this snapshot

| Observed issue | Implemented correction | Verification boundary |
|---|---|---|
| Model/helpers frequently reported busy | Report-specific reasoning controls and fresh request IDs after confirmed failures; the model and 4,000-token allowance were unchanged | Mocked regression and source contract; not proof of universal provider availability |
| Reported wrong goal links or missing dashboard habits | The data audit found existing links consistent; exact navigation/provenance and Plan+Today synchronization were strengthened | Domain/screen tests and recorded audit, not a claim that existing family data was relinked |
| Full plans could not accept another habit | Expose actual capacity and remove/restore controls; preserve the six-slot rule | Atomic acceptance/capacity tests |

| Observed issue | Implemented correction | Verification boundary |
|---|---|---|
| Dense cards hid useful detail | Compact goal grids and habit rows; details/research in larger dialogs | UI and browser evidence |
| Valid Coach replies showed zero credits and no automatic habits | Recognize refined success outcomes before settlement; preserve bound habit slots | Actual API/SSE plus disposable PostgreSQL regression with synthetic usage |
| Cloud tab still showed removed credit messages | Ordinary reload loaded the already-deployed corrected bundle | Public asset/image comparison and same-tab before/after UI check |

Sources: report reliability, plan/link repairs, capacity/UI repairs, credit release and browser follow-up.

## Recorded application verification

The latest code-changing release, before this documentation task, recorded:

- **1,073 backend tests passed**, with 100% statement and branch coverage across 5,091 statements and 1,234 branches; Ruff and mypy passed.
- **206 frontend tests passed**, plus TypeScript/Vite build and focused Coach/budget tests.
- Eleven release-boundary tests, image audit and full container/browser smoke passed.
- The new Coach regression verified retrieval cost, two useful Coach credits, a bound automatic slot, a three-habit batch, one batch credit and no extra dispatch/debit on replay.
- UI regression verified four goals, three choices per goal, two concurrent batches and meter updates without inline accounting text.
- All 824 packaged public assets matched source; container smoke used a disposable database and synthetic provider responses, then removed its synthetic resources.
- Both AWS service stacks completed and fourteen public checks verified deployed image/configuration identity, TLS, CORS, redirects, auth boundaries and DB readiness.

See regression evidence, container smoke, public verification and browser visibility investigation.

These are dated records. This documentation update does not claim to have rerun the full application suite or made paid model calls. Renamed evidence-output paths are checked as filesystem references; no browser scenario is run merely to rename its output folder.

## Documentation verification

The final integration checks cover:

1. Every original file remains under `docs-detailed`; screenshots and other binary evidence are byte-identical. Necessary text changes are navigation/path repairs and the historical-folder introduction.
2. README links to every capability document, each with a summary and key points.
3. New Markdown source links and cross-document paths resolve in the HQ workspace.
4. Mermaid blocks have balanced fences and valid syntax where a local renderer/parser is available; the recorded check states exactly what was exercised.
5. Active references to the renamed top-level folder are repaired, without changing `backend/docs`, public `/docs` routes or external providers' documentation URLs.
6. Independent reviewers compare AI call counts, model choices, data ownership, costs, catalog research and deployment boundaries across the documents.
7. No private environment payloads, credentials, vectors or family prose are copied.

The generated documentation verification record gives the final file/link/diagram counts and limitations. It is documentation evidence, not a deployment receipt.

## Known limits and next engineering checkpoints

| Area | Remaining boundary |
| --- | --- |
| End-user latency | No representative cloud p50/p95 for current full interactions; text-ready and all-habits-ready differ |
| Embedding quality | Small synthetic study and incomplete editorial labels; actual small/512 ingestion did not repeat the whole held-out benchmark |
| Research | Evidence-informed cards are not trials of this app or generated actions; adaptations/exclusions must remain visible |
| Economics | Pinecone account tariff/minimums and invoices are not reconciled; historical missing usage remains unknown |
| Cost visibility | Coach inline accounting is hidden; detailed visibility remains surface-specific elsewhere |
| Provider verification | Network/TLS readiness is not an authenticated model-generation or entitlement test |
| Availability | One task per service and Single-AZ micro RDS are demo choices, not a resilient multi-AZ production tier |
| Security maintenance | Recorded API base-image findings remain open; proxy trust, in-memory rate limits and data/privacy controls need review before scale-out |
| Database assurance | Restore-time ledger verification is recorded; runtime cannot independently read the operator ledger |
| Live identity | Automated release checks did not complete a new Google account sign-in; later signed-in browser observations are a different check |
| Browser freshness | Open production tabs require refresh to load a new bundle; there is no automatic update notification yet |

Address those checkpoints with a separately scoped implementation/evaluation plan. Do not silently expand this documentation task into paid benchmarks, model/index changes, database migrations, budget edits or cloud resource changes.